TECHNISCHE HOCHSCHULE DEGGENDORF
Campus Cham

Master's Thesis

Deggendorf Institute of Technology, Campus Cham
Faculty of Applied Natural Sciences and Industrial Engineering
Master Mechatronics & Cyber-Physical Systems

Automatische Programmerzeugung für einen Blechhandhabungsroboter vor einer Abkantpresse

Automatic Program Generation for a Sheet-Metal Handling Robot in Front of a Press Brake

Master's thesis to obtain the academic degree:
Master of Engineering (M.Eng.)

Submitted by: Sooraj Sudhakaran.
Matriculation Number: XXXXXXXX
First Examiner: Prof. Dr. Matthias Górka

Cham, March 28, 2026

Abstract

Acknowledgements

List of Tables

3.1 Functional Requirements for Automatic Robot Program Generation 24
3.2 Non-Functional Requirements — NFR-01 to NFR-04 with acceptance criteria and verification methods 26
3.3 Requirement Traceability Matrix — all fifteen requirements (FR-01 to FR-07, NFR-01 to NFR-05, SR-01 to SR-03) mapped to implementation modules and verification scenarios 33

4.1 Structural comparison of Concept A, Concept B, and Concept C across key design features 42
4.2 Structural comparison of Concept A, Concept B, and Concept C across key design features 45

5.1 Software modules across system layers and their primary responsibilities 53
5.2 Failure Mode and Effects Analysis (FMEA) of the robotic bending system 87

List of Figures

2.1 Robotic press brake bending system with integrated robot handling source -[18] 11
2.2 Teach pendant interface for manual robot programming (Source: Own work9) 14
2.3 Offline robot programming environment with CAD-based simulation (Source: [20]) 16

3.1 Tool Center Point parameters in Drag&Bot. 31

4.1 Virtual robotic environment used to evaluate the proposed concept (Drag & Bot platform). 40

5.1 System architecture 50
5.2 Five layer architecture 52
5.3 Virtual robotic environment used to evaluate the proposed concept within a simulated press brake cell (Drag & Bot platform). 55
5.4 Communication architecture of the implemented system, illustrating the interaction between the planning software, the drag&bot runtime environment, and the robot controller (adapted from [24]). 57
5.5 Robot control panel interface used for system operation and monitoring. 58
5.6 Virtual simulation environment used for implementation and validation. The setup includes the robot, press brake, orientation table, and sheet workpiece. Source: own work. 61
5.7 Sheet metal workpiece model used for geometry analysis. Internal cut-outs are detected and excluded from the grip candidate search region. Source: own work. 62
5.8 Operator Workflow .Every gate must be passed before the next begins.All operator interactions occu within the drag and bot enviornment. *Source: own work.* 63
5.9 TCP of UR5e robot. *Source: own work.* 65
5.10 Zone classification for robot workspace segmentation. *Source: own work.* 66

5.11 Fixture pose deviation validation workflow. Each monitored fixture is evaluated by comparing positional and angular deviations against defined thresholds. Out-of-tolerance or missing fixtures trigger a blocking warning dialog requiring operator confirmation. Validation is applied uniformly across all spatial zones. Source: own work. 69
5.12 Three-level system architecture illustrating the hierarchical structure of the implementation. *Source: own work.* 73
5.13 Grip position planning and feasibility validation workflow. The process begins with the loading of sheet geometry and gripping parameters, followed by vacuum holding feasibility validation. Candidate cup positions are then generated on the sheet surface, filtered using geometric constraints, and evaluated according to the configured scoring criteria. In the two-cup configuration, feasible candidates are additionally assessed as pairs before the highest-scoring grip configuration is selected. The final grip position is used to derive the approach, attachment, and retreat waypoints forwarded to trajectory generation. Source: own work. 77
5.14 Opposite face flip operation for regrip orientation adjustment . . 81

List of Abbreviations

OLP	:	Offline Programming
ROS	:	Robot Operating System
TCP	:	Tool Center Point
UI	:	User Interface
CAD	:	Computer-Aided Design
CAM	:	Computer-Aided Manufacturing
API	:	Application Programming Interface
DOF	:	Degree of Freedom
FR	:	Functional Requirements
NFR	:	Non-Functional Requirements
WSM	:	Weighted Sum Model
URDF	:	Unified Robot Description Format
YAML	:	YAML Ain’t Markup Language
Digital IO	:	Digital Input/Output
JSON	:	JavaScript Object Notation
SHA-512	:	Secure Hash Algorithm 512-bit
X	:	Translation along the X-axis (metres)
Y	:	Translation along the Y-axis (metres)
Z	:	Translation along the Z-axis (metres)
RX	:	Rotation about the X-axis (degrees)
RY	:	Rotation about the Y-axis (degrees)
RZ	:	Rotation about the Z-axis (degrees)
WebSocket	:	Web-based communication protocol for real-time data exchange defined in RFC 6455
LIN	:	Linear motion command
PTP	:	Point-to-point motion command
HI Dialog	:	Human Interaction (Drag&Bot-specific dialog service)
HTTP	:	Hypertext Transfer Protocol
HTTPS	:	Hypertext Transfer Protocol Secure
IP	:	Internet Protocol
DNB	:	Drag&Bot
UR5e	:	Universal Robots 5e (robot model)
STL	:	Standard Tessellation Language.

Contents

Abstract **I**

Acknowledgements **III**

List of Tables **V**

List of Figures **VIII**

List of Abbreviations **IX**

1 Introduction **1**
1.1 head of robot 1

2 State of the Art **2**
2.1 Manual And teach pendant based robot programming. 2
2.2 Offline programming approaches for industrial robots 3
2.3 Automatic and Rule-Based Robot Program Generation 4
2.4 Robotic Sheet-Metal Bending and Press-Brake Cells 5
2.5 Summary of the State of the Art and Identified Research Gap . 8
2.6 Fundamental of Industrial Robotics 9
2.7 Sheet Metal Bending and Press Brake systems 10
2.7.1 Process Characteristics and Challenges 11
2.7.2 Grasping and Vacuum-Based Handling 12
2.7.3 Trajectory Planning in Bending Environments 12
2.7.4 Limitations of Existing Industrial Systems 13
2.7.5 Relevance to This Thesis 13
2.8 Manual and Teach Pendant Programming 13
2.8.1 Limitations of Manual Programming 14
2.8.2 Challenges in Complex Manipulation Tasks 14
2.8.3 Transition Towards Automated Programming 15
2.9 Offline Programming Approaches for Industrial Robots 15
2.9.1 Digital Twin and Simulation-Based Programming 15
2.9.2 Limitations of Offline Programming 16
2.10 Automatic and Rule-Based Robot Program Generation 17
2.10.1 Task-Level Programming and Abstraction 17
2.10.2 Rule-Based and Constraint-Based Planning 18

3 System Concept and Methodology 20
3.1 Problem Definition and Objectives 20
3.2 Requirement Definition and Management 21
3.2.1 Functional Requirements 22
3.2.2 Non-functional Requirements 23
3.2.3 System Constraints and Assumptions 25
3.2.4 Software Platform and Tool requirement 28
3.2.4.1 SR-01 Drag And Bot runtime Platform 28
3.2.4.2 SR-02 Anybot Interface as Exclusive Communication layer. 29
3.2.4.3 SR-03 Tool Centre point. 30
3.2.5 System Traceability 31

4 Development of Concepts 34
4.1 Purpose and scope of concept development 34
4.2 Concept A – Structured operator Workflow with Guided Teach in programming 35
4.2.1 Methodological Framework 35
4.2.2 System Structure 36
4.2.3 Critical Evaluation 36
4.3 Concept B – Automatic Program Generation based on rules with Integrated Constraint Validation. 37
4.3.1 Methodological Framework 37
4.3.2 system structure 39
4.3.3 Critical evaluation 39
4.4 Concept C – CAD Integrated Offline Programming with Digital Twin Simulation 40
4.4.1 Methodological Framework 40
4.4.2 System Structure 41
4.4.3 Critical Evaluation 41
4.5 Structural Comparison of Concepts 42
4.6 Concept Decision-Weighed Sum Model (WSM) 43
4.6.1 Methodology 43
4.6.2 Evaluation Criteria and Weights 43
4.7 Result of the weighted Sum Model 45
4.8 Analysis of results and Concept Decision 45

5 Implementation 49
5.1 Introduction and scope 49
5.2 System architecture 51

5.2.1 Layered Architecture Design and data flow 51
5.2.2 Module Overview and dependency Discipline 53
5.2.3 Virtaul Enviornment setup 55
5.2.4 Communication Stack, Authentication interface Specification 55
5.2.5 Configuration Architecture and YAML File Structure 58
5.2.6 Operator interface – interactive 3d sheet bending viewer 60
5.3 Development and simulation Environment 63
5.3.1 Simulation Platform, Robot Model, and TCP Configuration 63
5.3.2 Virtual Cell Layout and Spatial Zone Classification 65
5.4 Pre-Execution Safety Validation 67
5.4.1 Fixture Pose Deviation Validation 67
5.4.2 Operator Authorization and Runtime Error Monitoring . . 68
5.5 Workpiece Orientation Strategy 70
5.5.1 Zone-Based RZ Admissibility 70
5.5.2 Pickup Orientation Validation 70
5.5.3 Three-Level Orientation Selection Strategy 70
5.6 Workpiece Handling and Gripper Control 73
5.6.1 Vacuum Gripper Digital IO Sequence 74
5.6.2 Kinematic Object Attachment and Detachment 74
5.7 Grip Position Planning and Feasibility Validation 75
5.7.1 Sheet Geometry and Hole Detection 75
5.7.2 Vacuum Holding Force Validation 75
5.7.3 Grip Candidate Search and Selection 76
5.8 Regrip Planning and Execution 77
5.8.1 Regrip Decision Strategy 78
5.8.2 Regrip Execution Sequence 79
5.8.3 System Integration and Configuration 79
5.9 Opposite-face Bending Workflow 80
5.9.1 Pre-Flip Verification and Transition Correction 80
5.9.2 Sheet Flip Sequence and Opposite-face bending Execution 80
5.10 Trajectory Generation 81
5.10.1 Three-Phase LIN Trajectory Structure 82
5.10.2 Waypoint Generation and Interpolation 82
5.10.3 Kinematic Mapping and Joint Execution 83
5.10.4 Workspace Constraints and Singularity Handling 83
5.10.5 Motion Execution and Path Reduction 83
5.11 System Integration, Execution Pipeline, and Conclusion 84
5.11.1 Conclusion 85

5.12 System Reliability and Failure Mode Analysis 85
5.12.1 Identified Failure Modes 86
5.12.2 Mitigation Strategies 88
5.12.3 Discussion 88

References **89**

1 Introduction

YSBASHFKJSDHFKfdghghjg

1.1 head of robot

XGFGF sooraj sudhakaran

2 State of the Art

2.1 Manual And teach pendant based robot programming.

Industrial robot programming has traditionally relied on teach pendant-based approaches, where robot motions are explicitly defined and programmed directly during operation. In this method, the programmer manually guides the robot to a sequence of target positions and records these positions using manufacturer-specific programming languages. This approach remains widely used due to its reliability and direct interaction with real-world systems [1].

However, teach pendant programming requires substantial expertise in robot kinematics, tooling, and process constraints. Even minor changes in part geometry, fixture placement, or cell layout often require extensive manual reprogramming, leading to increased commissioning effort and system downtime. Consequently, this approach scales poorly in flexible manufacturing environments characterised by frequent product changes and small batch sizes.

In robotic sheet metal handling and press brake applications, these limitations become more pronounced. Large and flexible workpieces introduce additional challenges related to reachability, collision avoidance, and process sequencing. Each step of the bending operation requires the programmer to manually define intermediate robot positions, making the process highly dependent on operator expertise. This often results in iterative trial-and-error adjustments during commissioning [2].

Furthermore, conventional teach pendant programming lacks the level of abstraction required for modern industrial applications that demand rapid reconfiguration and adaptability [1]. Programs created using this approach are typically tightly coupled to specific robot models, cell configurations, and individual programming styles, which reduces portability and maintainability.

As a result, manual teach pendant programming is generally unsuitable for advanced automation tasks that require high flexibility, integration of process knowledge, and reduced human intervention. These limitations highlight the need for higher-level and more automated programming approaches, which are discussed in the following sections.

2.2 Offline programming approaches for industrial robots

The limitations associated with manual teach pendant programming have led to the widespread adoption of offline programming approaches in industrial robotics. Offline programming enables the development and validation of robot programs within a virtual environment, using digital models of robots, tools, grippers, and workpieces, without interrupting ongoing production [1].

This approach is typically based on computer-aided design (CAD) and computer-aided manufacturing (CAM) models, which provide geometric representations of the robot, the workpiece, and the surrounding environment. These models allow robot trajectories and task sequences to be simulated and verified prior to deployment in the physical system. As a result, offline programming reduces commissioning time and improves safety by enabling early detection of potential issues such as reachability limitations and collisions [3].

In robot-assisted sheet metal handling and press brake operations, offline programming is commonly used to generate robot programs for tasks such as sheet handling, bending sequences, and intermediate regripping operations. Through virtual simulation, engineers can evaluate the feasibility of robot motions, validate bending sequences, and identify potential conflicts with tooling or fixtures [2].

Despite these advantages, offline programming has several limitations. Its effectiveness depends heavily on the accuracy of the digital model. Any discrepancy between the virtual model and the physical system, such as calibration errors, fixture misalignment, or changes in tool geometry, can lead to incorrect robot behaviour during execution. In such cases, additional manual adjustments and reprogramming are required during commissioning.

Furthermore, offline programming is less suitable for production environments with frequent changeovers and high product variability. Robot programs are typically developed for specific part geometries and process conditions, which limits their adaptability. In multi-stage bending operations, regripping and repositioning steps must still be manually defined and validated by engineers, increasing development effort and reducing flexibility [2], [3].

In summary, offline programming provides significant improvements over manual programming by enabling virtual validation and reducing on-site teaching effort. However, its reliance on accurate digital models and manually defined task sequences limits its effectiveness in highly flexible manufacturing scenarios. These limitations indicate the need for higher-level and

more automated robot programming approaches, which are discussed in the following section.

2.3 Automatic and Rule-Based Robot Program Generation

To further reduce the engineering effort associated with manual and offline programming, research in industrial robotics has increasingly focused on automatic robot program generation. These approaches aim to derive executable robot programs from high-level task descriptions and geometric information, thereby minimising the need for direct human intervention [1], [3].

A prominent approach within this domain is rule-based and task-level programming. In such methods, predefined rules or structured task descriptions are used to resolve constraints related to robot kinematics, tool orientation, and process feasibility. This enables the robot to execute recurring industrial tasks in a consistent and systematic manner while reducing operator involvement [3], [4].

Many systems adopt reusable motion primitives or skill-based representations as the foundation for automatic program generation. Instead of programming individual joint motions, higher-level actions such as grasping, approaching, or repositioning are defined as modular components. These components can be combined to form complex task sequences, improving modularity, scalability, and reusability in robot programming [4], [5].

Automatic program generation is often supported by geometric and feature-based representations of workpieces, typically derived from CAD models. These representations allow the system to automatically identify relevant features, such as edges, surfaces, or grasping regions, and to generate corresponding task sequences and approach strategies. Compared to manual and offline programming, this approach improves adaptability to different product variants and reduces programming effort [6].

These methods have also been applied to robotic sheet metal handling and press brake automation. In such systems, bending sequences, grasp configurations, and placement strategies can be derived from geometric constraints and process requirements rather than manual teaching. This reduces engineering effort and improves consistency for repetitive tasks involving similar workpieces [2], [7].

Despite these advantages, rule-based program generation approaches exhibit several limitations. Most systems rely on predefined rules and assumptions regarding part geometry, fixture layout, and process conditions,

which restrict their applicability in highly variable production scenarios. As task complexity increases, the number of rules also grows, making the system more difficult to maintain, scale, and validate [1], [4].

In the context of sheet metal bending, regripping operations remain a significant challenge. Multi-stage bending often requires the workpiece to be repositioned using an orientation table, followed by regrasping in a new configuration. Existing approaches typically rely on predefined regripping strategies tailored to specific tools and cell layouts, limiting their general applicability [2], [7].

Furthermore, although automatic grasp planning and trajectory generation methods have been investigated, they are often developed independently and are not fully integrated into a unified planning framework for industrial press brake systems. This lack of integration limits the ability to handle complex, multi-stage manipulation tasks involving both grasp planning and reorientation.

In addition to task-level planning, trajectory generation plays a critical role in translating high-level robot actions into executable motion. Industrial robot systems typically rely on interpolation-based trajectory generation, where motion between poses is discretised into intermediate waypoints to ensure smooth and continuous execution [1], [8]. These approaches ensure that the robot follows controlled Cartesian paths and avoids abrupt changes in motion. However, many industrial implementations treat trajectory generation as a low-level controller function, with limited integration into higher-level planning strategies. This separation can reduce flexibility when adapting to complex handling tasks such as regripping and multi-stage manipulation.

In summary, automatic and rule-based programming approaches significantly reduce engineering effort and improve consistency compared to traditional methods. However, their reliance on predefined rules and limited adaptability highlights the need for more integrated and flexible solutions, particularly for applications such as robotic sheet metal bending. These challenges are further examined in the following section.

2.4 Robotic Sheet-Metal Bending and Press-Brake Cells

Robotic press brake sheet metal bending has gained significant attention in industry due to the increasing demand for high productivity, repeatability, and consistent bending quality. In such systems, industrial robots are used to handle sheet metal workpieces throughout the bending process, includ-

ing gripping, positioning at the press brake, reorientation for subsequent bending operations, and unloading after process completion. Compared to manual operation, robotic systems reduce operator effort while improving process consistency and reliability [1].

Early research focused on the feasibility of integrating industrial robots with traditional press brake systems. These studies demonstrated that robotic sheet metal bending can be successfully implemented when robot motion planning is carefully synchronised with the bending sequence [2]. This highlighted the importance of coordinated motion planning, particularly for ensuring accurate positioning and safe interaction with press brake tooling.

Subsequent research addressed process planning challenges, including the determination of bending sequences, tool selection, and workpiece handling strategies. These studies showed that the feasibility of bending operations depends strongly on part geometry, robot reachability, and grasp configuration. As a result, planning methods must account for kinematic constraints and avoid configurations that lead to unstable grasps or infeasible robot motions [7].

Unlike rigid workpieces, sheet metal undergoes geometric changes during the bending process. This requires the robot to continuously adapt both the gripping position and the workpiece orientation throughout the sequence. To manage this complexity, many industrial systems rely on conservative motion strategies and predefined handling configurations, which simplify implementation but reduce flexibility [1], [2].

Regripping operations represent one of the most critical challenges in robotic sheet metal bending. During multi-stage bending, the workpiece must often be placed temporarily on an intermediate support, such as an orientation table, and then regrasped in a different configuration to enable subsequent bends. In most existing systems, these regripping operations are manually defined or predefined for specific setups, increasing engineering effort and limiting adaptability to new part geometries [2], [7].

More advanced approaches address regripping as a planning problem rather than a predefined operation. In such methods, the workpiece is represented as a set of grasp and orientation states, and feasible transitions between these states are evaluated using kinematic and geometric constraints. The regrip sequence is then determined automatically by identifying a sequence of feasible states that transforms the workpiece from its initial configuration to the required bending pose. However, these approaches introduce increased computational complexity and are rarely adopted in in-

dustrial press brake cells, where deterministic and predictable execution is preferred.

In addition, handling operations that require access to the opposite face of a workpiece introduce further complexity in robotic manipulation. In such scenarios, the workpiece cannot be reoriented solely through in-gripper motion and must be physically repositioned using intermediate placements and regrasping actions. This type of manipulation is commonly addressed in robotics through regrasp planning, where a sequence of grasp and placement operations is generated to transform an object from an initial pose to a desired target pose [9]. These approaches typically model object reorientation as a series of feasible grasp transitions connected through stable placements, enabling robots to achieve configurations that are not directly reachable.

However, most existing methods focus on general object manipulation in structured environments and do not explicitly address industrial process constraints such as limited workspace, fixture-based reorientation, and task-specific sequencing. In particular, the integration of physical flip operations using dedicated support structures, such as orientation or flip stations, remains insufficiently explored in practical robotic press brake systems. This highlights the need for application-specific solutions that combine regrasp planning with controlled environmental interaction to enable reliable and efficient opposite-side processing.

Another important challenge in robotic sheet metal handling is the determination of feasible grip positions. In many existing systems, grasp locations are either manually defined or derived from simplified geometric rules, without considering the combined effects of grasp stability, robot kinematics, and subsequent bending operations. Although research has explored automatic grasp planning methods [10], these approaches are often not fully integrated into industrial press brake systems and remain dependent on predefined assumptions. Consequently, the automatic determination of robust and adaptable grip positions remains limited, particularly in scenarios involving varying part geometries and multi-stage bending processes.

In addition, trajectory planning in press brake environments introduces further constraints due to the presence of tooling, machine structures, and limited workspace. Robot paths must be carefully planned to avoid collisions while maintaining feasible kinematic configurations throughout reorientation and regripping operations [11]. These constraints further increase the complexity of automated process planning.

Research has also investigated grasp stability, particularly for vacuum-

based grippers, where reliable contact between the suction cups and the sheet surface is essential. While models for vacuum gripping and grasp evaluation have been proposed, these are often tailored to specific gripper designs and are not fully integrated into complete press brake automation systems [12].

In industrial practice, robotic press brake systems perform effectively when part geometry, tooling, and process parameters are predefined. However, such systems rely heavily on accurate digital models, predefined handling strategies, and manually configured regripping operations. As product variability increases, system flexibility decreases, and reprogramming effort becomes significant.

Consequently, robotic press brake bending remains well suited for structured and repetitive production scenarios, but current systems still lack the flexibility required for highly variable manufacturing environments. However, existing systems do not provide a unified and practical solution that simultaneously addresses automatic grip position determination and adaptive regrip planning within industrial press brake environments. These limitations highlight the need for integrated and adaptive approaches that can automatically determine grip configurations and regrip sequences while maintaining the robustness required for industrial deployment.

2.5 Summary of the State of the Art and Identified Research Gap

The evolution of robot programming has progressed from manual teach pendant programming to offline programming and, more recently, to automatic and rule-based program generation approaches. Manual programming methods remain widely used due to their reliability and direct interaction with the physical system, but they require significant expertise and offer limited flexibility in dynamic production environments. Offline programming improves upon these limitations by enabling virtual validation and reducing on-site programming effort; however, it remains dependent on accurate digital models and still requires manual definition of task sequences.

Automatic and rule-based programming approaches have been developed to reduce engineering effort and increase consistency by enabling higher-level task descriptions and modular system design. While these methods improve scalability and reusability, they often rely on predefined rules and assumptions, which limit their adaptability in applications involving complex object handling and multi-stage manipulation.

In the context of robotic sheet metal bending and press brake automation,

these limitations become particularly evident. Workpiece handling tasks, including regripping and grip position planning, play a critical role in determining process feasibility. Existing systems largely depend on predefined regripping strategies and manually defined or simplified grip configurations. As a result, the number of required regrips is not determined dynamically, and grip positions are not consistently optimised with respect to kinematic feasibility, process constraints, and workpiece geometry.

Furthermore, the interaction between grasp planning, reorientation, and motion feasibility is often not addressed within a unified framework. Many approaches treat these components independently, which can lead to infeasible or suboptimal execution sequences in practical applications. This separation limits the ability of the system to adapt to varying part geometries and process conditions.

Consequently, there exists a gap between current robot programming approaches and the requirements of flexible robotic sheet metal bending systems. There is a need for integrated methods that can automatically determine feasible grip configurations and regrip sequences while maintaining computational efficiency and ensuring reliable execution within industrial environments.

This gap forms the foundation for the proposed approach presented in Chapter 5. The developed system addresses these limitations by combining automatic grip position determination with an adaptive regrip planning strategy, integrated within a unified planning pipeline for robotic press brake applications.

START ADDITON AL CONTENT

2.6 Fundamental of Industrial Robotics

Industrial robots play a central role in modern manufacturing systems by enabling high precision, repeatability, and efficiency in automated processes. According to the international standard ISO 8373, an industrial robot is defined as a reprogrammable, multipurpose manipulator programmable in three or more axes [13]. These systems are widely used across applications such as material handling, assembly, welding, and forming processes, including sheet metal bending. A typical industrial robot consists of multiple joints that provide degrees of freedom (DOF), allowing the manipulator to position its end-effector in three-dimensional space. The position and orientation of the end-effector are defined using the Tool Center Point (TCP), which represents the reference point for task execution. The relationship between joint

variables and the TCP pose is described through forward and inverse kinematics, which form the mathematical foundation for robot motion planning and control [14]. From a programming perspective, industrial robots can be controlled at different levels of abstraction. At the lowest level, motion is defined explicitly in joint space or Cartesian space by specifying waypoints and trajectories. This approach requires detailed knowledge of robot kinematics and workspace constraints. At higher levels of abstraction, robot programming can be categorized into manual programming, offline programming, and task-level programming [1]. Manual programming involves direct interaction with the robot using teach pendants, where operators define positions and movements explicitly. Offline programming (OLP) introduces the use of virtual environments and CAD-based models to simulate and validate robot programs before execution. Task-level programming further abstracts robot control by allowing users to specify high-level goals, which are then decomposed into executable motion sequences automatically [15]. In recent years, research has focused on increasing the level of abstraction in robot programming to reduce dependency on expert knowledge and improve scalability in flexible manufacturing systems. Approaches based on motion primitives, skill-based programming, and knowledge-driven frameworks have been proposed to bridge the gap between high-level task specifications and low-level robot execution [16]. These developments form the basis for modern automated robot programming systems and are particularly relevant in complex applications such as robotic sheet metal bending, where multiple stages of manipulation, reorientation, and constraint handling are required.

2.7 Sheet Metal Bending and Press Brake systems

Sheet metal bending is a fundamental manufacturing process in which a flat metal sheet is plastically deformed into a desired shape by applying force using a press brake. A typical press brake system consists of a punch and die set, where the sheet is positioned and bent along predefined lines. The bending angle and final geometry depend on factors such as tool geometry, material properties, and applied force [17]. Figure 2.1 illustrates a typical robotic press brake setup, where an industrial robot is integrated with the bending machine to automate handling operations.

In automated bending systems, industrial robots are used to perform tasks such as picking the sheet, positioning it within the press brake, reorienting it for subsequent bending operations, and unloading the final part. Robotic integration enables improved repeatability and reduces operator de-



Figure 2.1: Robotic press brake bending system with integrated robot handling source -[18]

pendency in high-volume production environments [2].

2.7.1 Process Characteristics and Challenges

A key characteristic of sheet metal bending is that the geometry of the workpiece changes after each bending operation. Unlike rigid object manipulation, the robot must continuously adapt to a dynamically evolving part shape. This directly affects, Grasp feasibility, Collision avoidance, Reachability constraints, Orientation planning. Studies have shown that the feasibility of bending operations strongly depends on geometric constraints and tool accessibility, requiring careful coordination between robot motion and bending sequences[7].

Another critical challenge is collision avoidance within constrained environments. The robot must operate in close proximity to the press brake tooling, which limits available workspace and increases the risk of collisions. Path planning methods must therefore consider both robot kinematics and environmental constraints[8]. Subsection Regripping and Workpiece Reorientation One of the most complex aspects of robotic sheet metal bending is the requirement for regripping operations. During multi-stage bending, the workpiece often needs to be repositioned to enable subsequent bends. This is typically achieved by placing the sheet at an intermediate location (e.g., orientation table) and re-grasping it in a new configuration. Regrasp planning has been extensively studied in robotics and is typically formulated

as a search problem over feasible grasp configurations and transitions between them [9]. However, in industrial bending applications, regripping is often implemented using predefined strategies, limiting flexibility when handling varying geometries [7]. In industrial bending applications, however, regripping is often implemented using predefined strategies due to process constraints. This limits flexibility and increases engineering effort when handling new geometries. As a result, regripping remains a major bottleneck in achieving fully automated bending systems [2], [7].

2.7.2 Grasping and Vacuum-Based Handling

In many sheet metal applications, vacuum grippers are used due to their ability to handle flat surfaces efficiently. However, grasp stability becomes critical, especially when handling large or flexible sheets. Research on vacuum-based gripping systems highlights the importance of modeling suction forces and ensuring sufficient contact area for stable manipulation[12]. Additionally, data-driven approaches to grasp synthesis have been proposed to improve robustness in uncertain environments [19]. In bending processes, grasp selection must also consider future operations, ensuring that the chosen grasp does not interfere with subsequent bends or tool access. This introduces a coupling between grasp planning and process planning.

2.7.3 Trajectory Planning in Bending Environments

Trajectory planning for robotic bending is significantly more complex than standard pick-and-place tasks due to workspace constraints and dynamic geometry changes. The robot must generate collision-free paths while maintaining feasible kinematic configurations.

Early research on robot-assisted bending highlighted the importance of path planning methods that account for both robot motion and process constraints[19]. These methods must ensure that the robot can safely move between intermediate positions, especially during reorientation and regripping operations. Classical trajectory generation methods provide a foundation for generating smooth and feasible robot motion [8]. However, these approaches often assume static environments and do not fully address the challenges introduced by changing workpiece geometry and constrained bending setups.

2.7.4 Limitations of Existing Industrial Systems

Despite significant progress, existing robotic bending systems exhibit several limitations such as , Heavy reliance on predefined handling strategies,Limited adaptability to varying geometries,High engineering effort for process setup,Lack of integration between grasp planning, regrip planning, and trajectory generation Many industrial systems use conservative motion planning to ensure safety, which reduces efficiency when handling complex parts. Additionally, most approaches treat grasping, planning, and execution as separate stages, leading to suboptimal performance.

2.7.5 Relevance to This Thesis

The challenges identified in robotic sheet metal bending,particularly dynamic geometry handling, regripping, and constrained trajectory planning,directly motivate the approach developed in this thesis. Unlike traditional systems, the proposed implementation integrates, Automated trajectory generation, Constraint-aware motion planning, Structured execution pipelines This enables more flexible and scalable automation of bending operations, addressing key limitations identified in the state of the art.

2.8 Manual and Teach Pendant Programming

Manual robot programming using teach pendants represents one of the earliest and most widely adopted approaches in industrial robotics. In this method, robot motions are explicitly defined by an operator through direct interaction with the robot controller. The operator either guides the robot to desired positions or manually inputs Cartesian or joint coordinates, which are stored as waypoints and executed sequentially by the control system. Figure 2.2 illustrates a typical teach pendant interface used for manual robot programming. This programming approach provides a high level of control and is particularly suitable for structured and repetitive tasks. It allows operators to directly verify robot motions during programming, reducing the risk of unexpected behavior during execution. As a result, manual programming remains widely used in industrial environments where tasks are stable and do not require frequent modification [1].

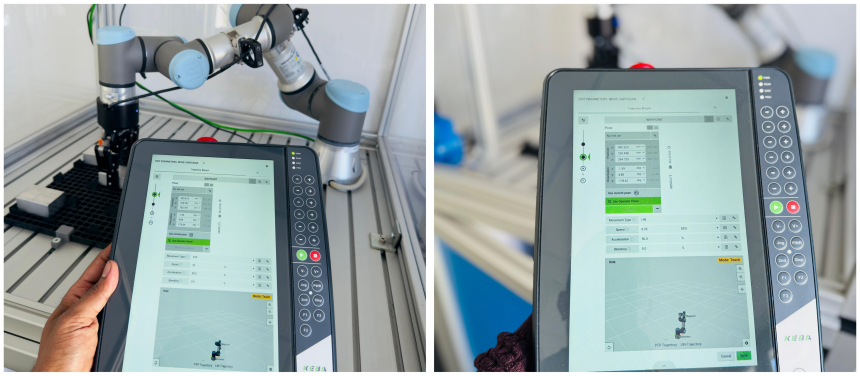


Figure 2.2: Teach pendant interface for manual robot programming (Source: Own work9)

2.8.1 Limitations of Manual Programming

Despite its advantages, manual programming exhibits several limitations, particularly in flexible manufacturing scenarios. One major drawback is the high dependency on operator expertise, as programming requires knowledge of robot kinematics, coordinate systems, and process constraints. This increases setup time and introduces variability depending on the skill level of the operator. Another limitation is the lack of scalability. When product geometry changes, the entire sequence of waypoints must often be redefined manually. This makes manual programming inefficient for applications involving high product variability or frequent reconfiguration [19]. Furthermore, manual programming does not inherently account for process constraints such as collision avoidance, reachability, or optimal path planning. These factors must be considered explicitly by the operator, increasing the complexity of programming and the likelihood of errors

2.8.2 Challenges in Complex Manipulation Tasks

In applications such as robotic sheet metal bending, manual programming becomes particularly challenging due to the need for multi-stage manipulation and reorientation. The operator must anticipate changes in workpiece geometry and define intermediate waypoints accordingly. Additionally, regripping operations require careful coordination of placement and grasping actions, which are difficult to define manually in a consistent and scalable manner. As task complexity increases, the number of required waypoints grows significantly, leading to longer programming times and reduced efficiency. These limitations highlight the inadequacy of manual programming for complex and dynamic industrial processes, where automation and adapt-

ability are required.

2.8.3 Transition Towards Automated Programming

The limitations of manual programming have motivated the development of higher-level programming approaches, including offline programming and automatic program generation. These approaches aim to reduce reliance on operator intervention by generating robot motions based on process requirements and system constraints. In modern industrial practice, there is a clear shift from manual programming towards automated and rule-based approaches, particularly in applications involving variable geometries and multi-stage processes. This transition forms the basis for the approaches discussed in the subsequent sections.

2.9 Offline Programming Approaches for Industrial Robots

Offline programming (OLP) is an established approach in industrial robotics that enables the development and validation of robot programs within a virtual environment prior to execution on physical systems. In contrast to manual programming, OLP decouples programming from production, allowing robot trajectories and task sequences to be generated without interrupting ongoing operations.

Figure 2.3 illustrates a typical offline programming environment, where a virtual model of the robot, tooling, and workcell is used to simulate and validate motion trajectories.

Offline programming systems typically rely on CAD models and geometric representations of the robot and its environment. These models are used to simulate robot motion, evaluate reachability, and detect potential collisions before execution. This approach significantly reduces downtime and improves safety by allowing errors to be identified during the planning phase [21].

2.9.1 Digital Twin and Simulation-Based Programming

A key development in offline programming is the use of digital twins, which represent a virtual replica of the physical robotic system. Digital twin models enable real-time synchronization between the simulated environment and the physical system, allowing more accurate validation of robot programs. Digital twin-driven manufacturing approaches have been widely studied, demonstrating their effectiveness in improving system reliability

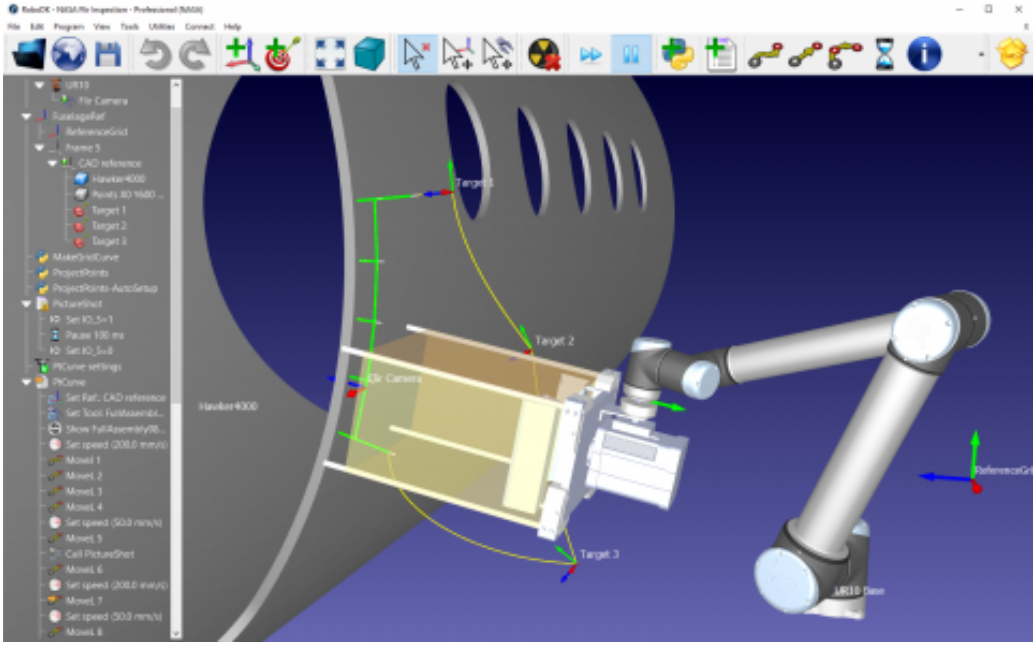


Figure 2.3: Offline robot programming environment with CAD-based simulation (Source: [20])

and reducing commissioning time [22]. By incorporating sensor data and system feedback, digital twins can enhance the accuracy of simulation-based planning. In industrial practice, simulation platforms such as Drag and bot RoboDK, ABB RobotStudio, and KUKA.Sim provide comprehensive environments for offline programming, including path planning, collision detection, and code generation. Advantages of Offline Programming compared to manual programming, which Reduced production downtime, as programming is performed outside the physical system, Improved safety due to collision detection and validation in simulation, Enhanced visualization, enabling operators to analyse robot motion before execution, Support for complex environments, where direct manual programming is difficult. These advantages make OLP a widely adopted solution in industrial automation, particularly for applications requiring precise and repeatable motion.

2.9.2 Limitations of Offline Programming

Despite its benefits, offline programming has several limitations. One major challenge is the dependency on model accuracy. Any mismatch between the virtual model and the real environment can result in deviations during execution, requiring manual correction. Furthermore, offline programming still relies heavily on manual input for defining trajectories and task sequences. Although simulation tools assist in validation, they do not inherently generate robot programs automatically. As a result, significant engineering effort

is still required to adapt programs to new workpiece geometries or process changes [19]. Another limitation is the lack of integration with higher-level task planning. In many systems, offline programming is used primarily for trajectory validation rather than for fully automated task generation. This limits its ability to handle dynamic and complex industrial processes such as robotic sheet metal bending. While offline programming improves efficiency compared to manual programming, it does not eliminate the need for human intervention in defining robot motions. In applications involving dynamic geometry changes and multi-stage manipulation, such as sheet metal bending, these limitations become significant. The approach developed in this thesis builds upon the advantages of simulation-based environments while addressing their limitations by introducing automated trajectory generation and constraint-aware motion planning. Unlike traditional offline programming, the proposed system generates robot trajectories programmatically and executes them through a structured pipeline, reducing reliance on manual programming.

2.10 Automatic and Rule-Based Robot Program Generation

Automatic robot programming aims to reduce or eliminate manual intervention by generating robot programs directly from high-level task descriptions, process parameters, or environmental inputs. Unlike manual and offline programming approaches, which require explicit definition of trajectories, automatic programming systems compute robot motions programmatically based on predefined rules, constraints, or models.

2.10.1 Task-Level Programming and Abstraction

A fundamental concept in automatic robot programming is task-level programming, where users specify what the robot should achieve rather than how it should move. The system then decomposes high-level tasks into sequences of executable actions. Early work on robot programming introduced the concept of abstraction layers, enabling robot programs to be generated from symbolic descriptions of tasks [15]. This approach reduces the need for detailed motion specification and allows systems to operate at a higher level of reasoning. More recent research has focused on skill-based programming, where tasks are represented as reusable motion primitives or skills. These skills encapsulate common operations such as grasping, placing, or approaching, and can be combined to form complex task sequences [16].

This modular approach improves scalability and reusability in industrial applications.

2.10.2 Rule-Based and Constraint-Based Planning

In structured industrial environments, automatic programming is often implemented using rule-based systems, where robot behaviour is defined through a set of logical rules and constraints. These rules govern decisions such as grasp selection, motion sequencing, and collision avoidance. Constraint-based approaches extend this concept by incorporating geometric, kinematic, and process constraints into the planning process. The system evaluates feasible solutions based on these constraints and generates valid robot trajectories accordingly. Such approaches are particularly effective in applications where process conditions are well-defined, enabling deterministic and reliable program generation. However, the complexity of rule sets increases significantly with task variability, making system design and maintenance more challenging [23].

A key challenge in automatic robot programming is the integration of multiple planning components, including Grasp planning, Regrip planning, Trajectory generation, Collision avoidance. In many existing systems, these components are developed independently, resulting in fragmented workflows and limited adaptability. For example, grasp planning methods may not consider future trajectory constraints, leading to infeasible execution scenarios. Research in robotic manipulation highlights the importance of integrating these components into a unified framework to achieve robust and efficient task execution [10]. However, achieving such integration in industrial environments remains an open challenge due to the complexity of real-world constraints. Despite significant advancements, current automatic programming approaches exhibit several limitations: such as, Limited integration between planning modules, grip position finding for changing workpiece geometry, collsion avoidance, Scalability issues in rule-based systems, Dependency on structured environments, Insufficient adaptability to dynamic geometry changes. These limitations are particularly evident in applications such as robotic sheet metal bending, where the workpiece geometry evolves during the process and requires continuous adaptation of robot actions. The limitations identified in automatic robot programming directly motivate the approach developed in this thesis. While existing methods provide partial automation, they do not fully integrate all required planning components into a unified pipeline. The proposed system addresses

this gap by, combining trajectory generation, constraint handling, and execution into a structured workflow, generating robot motions automatically from validated target poses, Reducing reliance on manual programming and predefined rules This integrated approach enables more flexible and scalable automation, particularly in complex applications such as robotic sheet metal bending.

3 System Concept and Methodology

3.1 Problem Definition and Objectives

The state of the art has shown clearly that industrial robots are widely used for the press brake sheet metal bending, primarily through manual programming or predefined programming methods. These methods perform well in structured scenarios with fixed part position, geometries, and bending sequences. However, as product variants increase, the adoption of such robot programming approaches requires manual reprogramming and time consuming.

In press brake bending, after each bending operation, the shape of the sheet metal changes. Each geometric change affects robot reachability, gripping feasibility and grip point calculations, motion trajectories. Consequently, handling and regripping should be planned carefully. Previous research shows that managing this constraint is one of the most time-consuming and complex aspects of robotic bending processes particularly when handled manually.

From a methodological perspective, the limitations of this robot programming approaches related to their low-level nature. programmers must define the robot paths, position of tools and individual waypoints. This makes the programming, tightly coupled to a specific robot, grip positions or tool setups and making the reorientation or regripping strategies difficult to adapt. Robotic literature clearly indicates that such low-level programming does not scale well as system complexity increases and flexibility requirements grow [1]. This observation motivated research into the higher-level representation, where motion execution and decision making integrated with a unified framework. These methods aim to simplify handling constraints and promote reusable robot programs. While these strategies have been implemented in other robotic applications, they have not yet been implemented in robotic press brake sheet metal bending, particularly when the position and geometry changes must be explicitly considered.

Based on these observation , the central problem addressed in this thesis can be defined as follow: There is a lack of a systematic approach for automatic generation of robot programs for press brake sheet metal bending that explicitly accounts for changing workpiece positions and geometry , handling feasibility and regripping requirements while minimizing the man-

ual programming .

The primary objective of this thesis is to develop a conceptual and methodological framework for the automatic generation of the robot program that integrates bending, handling and regripping operations in one workflow. This proposed method aims to reduce manual programming effort and improve adaptability to different part geometries. To achieve this objective ,the work focuses on integrating bending sequences , handling actions , opposite side bending and regripping operation and which are validated and verified in simulation environment compatible with industrial robot environment .The scope of the thesis is limited to virtual environment and conceptual evaluation.

3.2 Requirement Definition and Management

To ensure a systematic and verifiable development of the proposed automatic robot program generation approach, the system design is guided by explicitly defined requirements. These requirements turn the research gap and problem formulation into engineering objectives that can later be verified and validated. The requirement that are explained in this chapter are directly derived from the limitations identified in the state of art and from the objectives formulated in 3.1. Rather than describing implementation details , the requirement specify what the system must achieve under which condition it must operate .This separation ensures that design decision remain traceable and that verification activities can be performed objectively. The requirement are classified into functional and non-functional requirements .Functional requirements describe the expected system behavior and capabilities , while nonfunctional requirements define the quality attributes such as usability , robustness and safety .In addition , system constraints and assumptions are explicitly stated to scope of the work and to define the operational boundaries of the proposed solution To support systematic verification, each requirement is formulated to Specific measurable Achievable Relevant Timebound (SMART) principle. Furthermore, each requirement is linked to system components and verifications steps. This traceability ensures that all requirements can be checked later and that the development process remains transparent and consistent.

3.2.1 Functional Requirements

Functional requirements specify the capabilities that the system provides in order to address the problem of automatic robot program generation for press brake sheet metal bending. These requirements define the expected system behavior from the task execution, operator interaction, trajectory generation and recovery mechanism. At a high level, the system shall enable the automatic generation of robot programs for multi-step sheet metal bending process without requiring manual t programming. The system shall support the execution of bending sequences that involve changes in workspace geometry, orientation, handling and regripping strategies.

A central functional aspects of the system is the consideration of work piece geometry evolution during bending process .Since the shape , position , orientation of the sheet metal changes after each bending operations , the system shall be feasible to handle the current geometric state and position of the workpiece when generating subsequent actions for the next task. This requirement is essential to ensure feasibility with respect to robot reachability and tool operations throughout bending sequence. Furthermore , the system shall support bending operations that requires for different sides of the work piece .Opposite side bending introduces additional constraint on tool orientation and robot configuration , which cannot always be resolved with a fixed handling strategy and time consuming .The system program therefore capable of adapting this strategy , including workpiece orientation and grip configuration, to enable such operation while maintaining feasibility

To structure the functional requirements, they are grouped into categories.

Operator Interaction Requirements: The system shall allow limited operator interaction for configuration of robot and process selection while minimizing the manual intervention during the execution. Before execution, the operator shall be able to specify the required process output, such side for bending or cad model for bending. During execution, the robot operation shall provide clear feedback or warning message when the given output conditions are unsafe for the process. The role of operator shall be restricted to configuration, while the program generation logic shall be handled automatically by the system.

Data Handling Requirements: The system should automatically use the operator input data and process the program that need to processed for

the required output in a consistent and reliable manner .This includes Handling geometries , grip position calculation, tracked poses , configuration parameters and intermediate state information generated during the time of execution. Data shall be used consistently across different stage of the program generation process to ensure that decisions regarding the handling and orientation and regripping are based on the current system state.

Motion Execution Requirements: The system shall generate feasible robot waypoints for all handling and bending related operations. This includes generation of approach waypoints, placement motions and the execution of orientation changes of the work piece and tool. Trajectory plans should be calculated prior to execution to prevent unreachable target poses. In addition, the system shall determine and apply appropriate grip positions on the workpiece to support handling and picking from different bending configurations. For opposite side bending operations, the system shall identify the grip position and waypoints such that required tool alignment can be achieved without violating the kinematical limits. Grip selection shall be treated as an integral part of the program generation process rather than manual integrated adjustments.

Error Handling and Regripping Requirements: The system shall be capable of detecting infeasible target poses or unsafe execution conditions before execution. When such conditions are detected, the system shall initiate a warning message to the user, trigger appropriate recovery actions instead of continuing execution. Recovery actions may include regripping or repositioning the workpiece to restore reachability. Regripping operation shall be supported as functional capability of the system .The system shall be able to integrate regripping sequence into generated robot program when direct trajectories to the target are unreachable or unsafe .If a critical error cannot be resolved through recovery actions , the system shall terminate execution in controlled regripping or repositioning manner to prevent unsafe robot behavior.

3.2.2 Non-functional Requirements

NFR defines the quality attributes and operation characteristics that elaborates how well the system performs its functions. While the functional requirements specify the system capabilities, non-functional requirements describe constraints related to usability, maintainability, robustness and safety.

Table 3.1: Functional Requirements for Automatic Robot Program Generation

ID	Functional Requirement (SMART)	Acceptance Criteria	Verification Method
FR-01	The system shall detect unreachable target poses before robot motion execution.	The robot shall not execute motion to unreachable poses and shall provide an error message.	Simulation based
FR-02	The system shall allow operator-side selection prior to program execution.	The selected bend side shall be applied correctly to the generated robot program.	Simulation based
FR-03	The system shall execute a regripping sequence when a target pose is unreachable within the current bend side.	Regripping shall be performed successfully and execution shall continue without interruption.	Simulation based
FR-04	The system shall handle workpiece orientation changes during program execution.	Orientation changes shall be applied correctly without violating kinematic limits.	Simulation based
FR-05	The system shall support opposite-side bending operations when required.	The system shall correctly execute opposite-side bending operations.	Simulation based
FR-06	The system shall determine and apply suitable grip positions for each workpiece state.	Grip positions shall be determined and applied correctly for each workpiece state.	Simulation based
FR-07	The system shall terminate execution when an unsafe operation is detected.	The robot shall terminate execution and display an error message.	Simulation based

These aspects are important for automated robotic systems, where corrected system behavior alone is not sufficient if the system is difficult to use, fails under complex variations or behaves unsafely during execution. The NFR of the system are derived from the operational context of press brake sheet metal bending, where industrial robot operates in close proximity to tooling, workpieces and cell boundaries. The system is intended to support flexible manufacturing scenarios with minimal operator programming while maintaining the user conditions and safe behavior. To ensure clarity, the NFRs are grouped into the following categories.

Usability Requirements: The system should be designed such that the user can operate the system easily by minimizing the operator interaction during normal operation. Configuration steps required prior to execution shall be limited to essential parameters and shall be presented in a clear and understandable manner. The system should give meaningful feedback to operator, including warning information and error messages when unreachable waypoints or unsafe condition are detected. These messages shall support the operator in understanding the cause of an issue without requiring expert knowledge of robot programming.

Maintainability Requirements: The system shall support maintainability by enabling configuration changes without requiring modifications to the source code. Process parameters, handling strategies and system setting shall be adjustable through configuration mechanisms that are separated from the core program logic. In addition, the system should provide sufficient logging or terminal output to support debugging, monitoring and future extensions. These measures are intended to facilitate adaptation of the system to new workpiece, process variations, with minimal programming effort.

Robustness Requirements: The system shall exhibit robust behavior under repeated execution cycles. Small variations in workpiece, orientation or exceptions timing or path shall not lead to system failure or unsafe behavior. The system shall handle such behavior deviation in controlled manner, ensuring smooth operations whenever feasible or triggering appropriate recovery actions when required. Robustness is particularly important in flexible manufacturing environments where idealized conditions cannot be assumed.

Safety Requirements: The system should always ensure safe operations by validating generated robot motion trajectories, informing the user about unsafe conditions and enforcing defined safety constraints and forbidden zones to prevent unreachable or unsafe poses during program execution. In the events of critical errors or unsafe conditions that cannot be resolved through recovery actions, the system shall terminate execution in a controlled manner.

3.2.3 System Constraints and Assumptions

In addition to functional and non-functional requirements, the development of the proposed automatic robot program generation system is subject to a defined sets of constraints and assumptions. These Constraints delimit the scope of the thesis and clarify the operational boundaries within which the system is designed to function. Explicitly stating these limitations ensures transparency and prevents misinterpretation of the intended system capabilities. The proposed approach is developed for a predefined robotic press brake cell configuration. The mechanical layout of the robot, press brake, tooling and auxiliary components is assumed to remain fixed throughout the system operations. Changes to physical robotic position and tool geometry, and non-collision movement planning are out of scope of this work. The system assumes the availability of accurate geometrical models of the robot,

Table 3.2: Non-Functional Requirements — NFR-01 to NFR-04 with acceptance criteria and verification methods

ID	Category	Non-Functional Requirement	Acceptance Criteria	Verification Method
NFR-01	Usability	The system shall minimise operator interaction during normal execution. Configuration steps prior to execution shall be limited to essential parameters. The system shall provide meaningful warning and error messages that do not require expert robot programming knowledge.	Operator interaction limited to bend side selection and execution confirmation. Warning messages displayed for all detected unsafe conditions.	Usability validation
NFR-02	Maintainability	The system shall allow process parameters, handling strategies, and system settings to be modified through configuration mechanisms that are fully separated from the core program logic, with no source code changes required. Sufficient terminal logging shall be provided to support systematic debugging, monitoring, and future extension of the system.	New workpiece geometry configurable via YAML file update only, without source code modification. Configuration change validated without rebuild.	Configuration validation test
NFR-03	Robustness	The system shall exhibit robust behaviour under repeated execution cycles. Small variations in workpiece orientation or timing shall not lead to system failure or unsafe behaviour. The system shall handle deviations in a controlled manner.	System completes at least five consecutive bending cycles without failure under nominal variation conditions.	Repeated run validation test
NFR-04	Robustness	The system shall tolerate moderate pose deviations within defined limits without triggering an error or aborting execution. Deviations exceeding the threshold shall trigger a warning and controlled stop.	Pose deviations within the defined tolerance threshold do not cause execution termination. Deviations exceeding the threshold trigger a warning and controlled stop.	Perturbation validation test

workpiece, press brake, and related components within the virtual environments. The correctness of reachability analysis and motion feasibility check depends on the validaty of these models. Model inaccuracies, calibration errors or unmodeled physical effects are not explicitly compensated within the proposed framework. Explicitly defining these limitations ensures transparency and prevents over interpretation of the system capabilities. The constraints are grouped into geometric, process related, cell layout, handling and safety limitations.

Sheet geometry Constraints : The system is designed for rectangular sheet metal workpiece only. All geometric reasoning, grip position calculations and edge distance check assumes a rectangular shape with four orthogonal edges. Nonrectangular geometries, polygonal shapes or free form contours are not supported. The sheet geometry is parameterized by its length, width thickness and mass. Optional circular holes may be defined using radial parameters. However, only circular holes are supported, irregular polygonal cuts are not considered. All hole's related checks are performed using radial distance evaluation. Furthermore, only a single global wall height parameter is considered during the handling. Per edge height customization is not supported.

Bend and process constraints: The bending process is restricted to 90 degrees of bend angles. Variable user defined bend angles are not supported. The bending logic assumes one bend per edge and each edge can be associated with at most one bending operation. A maximum of four bends per side is supported. For opposite side operation, the number of bends is similarly limited to four. The program logic prevents the duplicate bend assignments and multiple bend assignments on the same edge. This ensures a consistent one bend per edge configuration and simplifies the geometric reasoning required for the automatic program generation.

Robot and cell layout constraints: The robotic cell configuration is assumed to be fixed. The robot base, press brake, fixtures and auxiliary components are defined relative to a constant world coordinate frame. Relocation or reconfiguration of these elements invalidates the area logic, reachability assumptions and orientation constraints. In this case, system requires manual updates of configuration values for adaptation. Tool configuration is also assumed to be fixed. The system operates with a predefined TCP (tool center point) and attachment frame (Flange) Changes tool geometry, TCP definition or IO wiring require configuration updates and may necessitate software adaptation. Vacuum gripping is controlled via single digital output. Alternative control schemes are not considered.

Gripping constraints: The system is designed exclusively for vacuum based gripping. Grip feasibility checks are based on physical considerations including pressure difference friction and rated shear force. The holding capability of a vacuum gripper depends primarily on the achievable pressure

difference between suction cup and the ambient atmosphere, the effective contact area and the frictional interaction between workpieces surfaces and the suction elements.[12]. Mechanical magnetic or finger-based grippers are not supported within the current framework.

Safety and warning system constraints: The safety and warning logic monitors only a predefined ser of fixtures and reference objects within the cells. Full collision detection, dynamic safety trajectory planning and advanced real time safety validation are not implemented. Safety evaluation is limited to predefined geometric checks and rule-based constraints within virtual environment. Object tracking and naming assumptions -The systems rely on predefined object identifiers within the virtual environment. Specific object names are expected as key to finding to process or finding the objects. Renaming or removing these objects disrupts the workflow logic and may lead to execution failure. Therefore, renaming objects requires manual updates in the program configuration.

3.2.4 Software Platform and Tool requirement

Instead of coming from the functional analysis, the development context imposes three additional requirements. The industrial partner environment and the hardware available within the thesis scope determine the choice of execution platform, the communication interface that allows the planning system to reach the robot runtime, and the physical tool geometry from which all motion targets are computed. As a result, these limitations are formalized as SR-01, SR-02, and SR-03 software and tool platform requirements.

3.2.4.1 SR-01 Drag And Bot runtime Platform

The drag and bot software platform, developed by drag and bot GMBH. The drag and bot runtime will receive and carry out all robot motion commands, service calls, and object management operations produced by the planning modules; this work does not support any other execution environment. Two main requirements of this thesis serve as the foundation for choosing drag and bot as the execution environment. First, as mentioned in Section 3.1, the work is limited to a virtual environment, and drag and bot offers an integrated simulation runtime that enables thorough validation of robot programs without requiring access to actual robot hardware [24]. Second, the

platform allows the planning system to remain robot-independent at the software level by exposing a uniform application programming interface that abstracts manufacturer-specific robot controller syntax. By executing within the drag and bot virtual environment, the object tracking subsystem, the digital IO interface, and the object attach/detach services all operate under conditions representative of the target press brake cell, ensuring that simulation-based verification as specified for FR-01 through FR-07 is meaningful and reproducible. The suitability of runtime abstraction layers for controller-independent robot program generation is established in the modular robotic automation literature[25]. The drag and bot platform documentation confirms the availability of the virtual simulation environment, the AnyRobot Interface, and the RosBridge communication server on which the system depends. All planning modules developed are implemented exclusively against this API, and all validation test cases are executed within this environment.

Acceptance Criteria — SR-01: the planning system shall establish a stable authenticated connection to the drag and bot runtime within 5 seconds of launch; all motion, service, and IO commands shall be dispatched and acknowledged without error during the simulation

3.2.4.2 SR-02 Anybot Interface as Exclusive Communication layer.

The AnyRobot Interface, provided by drag and bot GmbH as part of the drag and bot software stack, shall constitute the sole communication channel between the planning system and the drag and bot runtime [24]. All robot motion commands: digital IO operations, object manager service calls, and operator requests shall be transmitted exclusively through this interface; direct controller-level communication or any alternative channel is outside the scope of this work. Because it combines all robot functions into a single WebSocket channel based on the RosBridge v2 protocol, such as LIN and PTP motion commands, vacuum gripper IO, kinematic object attachment, and modal operator dialogs, the AnyRobot Interface is widely used. Without a locally installed ROS environment, messages are sent as JSON-formatted objects that can be created and sent in Python. This architecture directly addresses the NFR-02 maintainability requirement by eliminating the need for a robot-manufacturer-specific post-processor and enabling the planning system to target a different robot model with just a configuration update. Connection security is enforced through a SHA-512-based rosauth authen-

tication handshake at session establishment, ensuring that only the authorized planning client can issue commands to the runtime - a design decision that supports the safety and robustness requirements NFR-03 and NFR-05.

Acceptance Criteria — SR-02: all planning-to-runtime communication shall occur exclusively over the AnyRobot WebSocket channel; an authenticated connection shall be established before any command is dispatched; LIN motion, digital IO, object manager commands shall each be successfully executed and acknowledged during the simulation

3.2.4.3 SR-03 Tool Centre point.

The tool center point (TCP) applied to all robot motion targets throughout the planning system shall be defined by the tool configuration registered in drag and bot. The TCP pose relative to the robot flange, expressed in the EULER_INTRINSIC_ZYX convention [26], is $x = 0.000\,\text{m}$, $y = 0.034\,\text{m}$, $z = 0.194\,\text{m}$, $Rz = -90.0°$, $Ry = -75.0°$, $Rx = 0.0°$. These values define the geometric offset from the robot flange frame to the contact surface center of the vacuum suction cup mounted on the end-effect. The tool center point in the world coordinate frame is where all motion targets generated by the planning system are expressed, including pickup poses, pre-bend approach poses, regrip placement poses, and post-bend unload poses. The inverse of the flange-to-TCP transform is used to derive the corresponding flange pose from the TCP target, which is then executed by the robot controller [14]. A positioning error at the workpiece contact surface results from any mismatch between the declared TCP offset and the actual gripper geometry. Such an error would jeopardize the viability of the grip and increase the risk of workpiece slippage during handling in the context of vacuum gripping, where holding force depends on the achievable pressure differential and the effective contact area [12]. By defining the TCP as a formal, traceable requirement instead of an implicit code constant, design transparency is maintained and future tool replacements are reflected as intentional requirement changes. As seen in fig 3.1 , the TCP pose values are registered in the drag and bot tool management system as project_3_tcp and are obtained from the physical geometry of the suction gripper assembly installed on the robot within the press brake cell. In accordance with the configuration-driven design required by NFR-02, these values are loaded from the input configuration file at runtime.

Acceptance Criteria - SR-03: all motion targets generated during simulation test execution shall produce flange poses consistent with the declared project_3_tcp offset; the TCP identifier and pose values shall be loaded exclusively from the YAML configuration file; a change to the TCP pose in the configuration file shall propagate correctly to all generated motion targets without requiring source code modification.

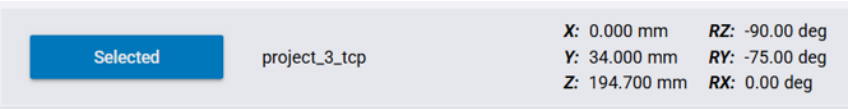


Figure 3.1: Tool Center Point parameters in Drag&Bot.

3.2.5 System Traceability

Requirements Traceability is a fundamental principle in systematic engineering development, ensuring that all defined requirements are consistently reflected in the system design, implementation and validation stages. In the context of the proposed automatic robot program generation system, traceability guarantees that the transition from conceptual objectives to executable functionality remains transparent, structured and verifiable. Each requirement is defined in section 3.2.1 (functional requirement) and Section 3.2.2(Nonfunctional requirements) is assigned to a unique identifier (e.g. FR-01, NFR -019:) .These identifiers serve as persistent reference throughout the entire development lifecycle. The identifiers are used consistently in the following aspects:

- Every requirement is implemented within a specific module.
- Every requirement is verified through a defined test scenario.
- No requirement remains unnoticed.
- No implementation functionality exists without requirement justification.

Forward traceability: confirms that every requirement is realized in a specific system components or logic module.

Backward traceability: confirms that every implemented functionality can be tracked back to an explicitly defined requirement.
By mapping each requirement to a corresponding implementation module

and validation scenario , the development process become auditable and reproducible, which is particularly important in systems involving automated decision making , such as traceability evaluation , regripping logic , and opposite side handling .In addition tractability improves maintainability and future extensibility, such as adapting to a new gripper type or extending bend angle support . This structure allows systematic impact analysis. Affected requirements and modules can be identified quickly, thereby reducing development risk.

Table 3.3: Requirement Traceability Matrix — all fifteen requirements (FR-01 to FR-07, NFR-01 to NFR-05, SR-01 to SR-03) mapped to implementation modules and verification scenarios

Req. ID	Requirement Description	Implemented In	Verified In
FR-01	Detect unreachable target poses before execution.	Reachability checking and admissibility validation logic	Simulation test case TC-01
FR-02	Allow operator-side selection of bending configuration before execution.	Operator dialog and configuration collection logic	Execution test case TC-02
FR-03	Execute regripping sequence when a target pose is unreachable.	Automatic regrip cycle logic with I/O and attach/detach control	Regrip validation test case TC-03
FR-04	Handle workpiece orientation changes during program execution.	Orientation conversion, rotation normalisation, and strategy selection logic	Orientation validation test case TC-04
FR-05	Support opposite-side bending operations when required.	Opposite-side bending workflow with sheet-flip and regrip completion logic	Opposite-side validation test case TC-05
FR-06	Determine and apply suitable grip positions for each workpiece state.	Grip planning logic with edge-clearance, spacing, and vacuum-force feasibility checks	Grip validation test case TC-06
FR-07	Terminate execution when an unsafe operation is detected.	Safety warning and pre-execution deviation checking logic with operator confirmation dialog	Safety validation test case TC-07
NFR-01	Ensure minimal operator interaction during normal execution.	Operator interaction limited to bend-side selection and execution confirmation	Usability validation
NFR-02	Allow process parameter changes without source code modification.	Configuration reader and YAML-based single-source process configuration	Configuration validation test
NFR-03	Provide clear warning and error messages for unsafe conditions.	Warning generation and modal operator notification logic	Warning message validation
NFR-04	Ensure robustness under repeated execution cycles.	Deterministic execution loop with automatic waypoint generation and path logging	Repeated run validation test
NFR-05	Tolerate moderate pose deviations within defined tolerance limits.	Configurable pose-deviation threshold and perturbation handling logic	Perturbation validation test
SR-01	drag&bot GmbH runtime platform shall be the exclusive robot execution environment.	Runtime client connection logic for communication with the configured drag&bot host and port	Platform integration check: runtime connection and command dispatch verified
SR-02	AirRobot Interface shall be the exclusive software communication layer between planning system and robot runtime.	Authenticated WebSocket communication layer with JSON command exchange	Interface integration check: authenticated connection and JSON command round-trip verified
SR-03	TCP shall be configured as project_3_tcp: $x = 0.000$ m, $y = 0.034$ m, $z = 0.194$ m, $R_x = -90.0°$, $R_y = -50.0°$, $R_z = 0.0°$.	Default TCP pose definition applied in all flange-to-TCP transformations	TCP configuration check: flange pose computed correctly for reference target poses

4 Development of Concepts

4.1 Purpose and scope of concept development

Concept development is an essential part of structured design, which consists of the design phase where different principles solutions are developed and evaluated before implementation. This is an essential part of systematic engineering design. By evaluating different options with respect to certain criteria before proceeding, the chances of selecting an inappropriate design path are minimized. REF The concept development phase is crucial for the success of the project, as it allows for the exploration of different design alternatives and the selection of the most suitable one based on a structured evaluation process.

The gap in the research, as identified in Section 2.5, reveals the absence of an effective systematic approach to automatically generate robot programs for press brake sheet metal bending. This approach should consider changes in workpiece geometry, gripping, re-gripping, and pre-execution pose, minimizing manual programming. The gap in the research is addressed in the form of measurable engineering objectives, as discussed in 3, with respect to functional requirements FR-01 to FR-07 and non-functional requirements (NFR-01 to NFR-05), which constrain how it must behave; and software platform requirements SR-01 to SR-03 (Table 3.3), which are fixed by the industrial partner environment and therefore non-negotiable. Three conceptually distinct approaches are developed in this chapter because the research gap can plausibly be addressed by different paradigms, and the selection must be justified rather than assumed

Three solution principles are proposed. The first, designated Concept A, proposes a guided teaching approach in which a structured software workflow supports the operator during conventional manual robot programming. Concept B proposes an automatic program generation system in which waypoint, grip positions , regrip sequences and opposite side handling strategies are derived automatically by evaluating the inputs configuration file , with all communication to the robot runtime conducted via the AnyRobot Interface over an authenticated WebSocket channel as required by SR-01 and SR-02.[24] Concept C is a CAD integrated offline programming in which a digital cell model is used to generate , validate and execute robot program within a simulation environment [1],[3]

The concept decision is made using Weighted Sum Model (WSM), a quantitative decision support method in which evaluation criteria are assigned relative weights and each concept is scored against criterion. This method satisfies the requirement that concept selection must be based on structure rather than on qualitative judgment. The morphological box and preferences analysis methods are alternative methods, The WSM is selected because it enables simultaneous multi-criteria weighting and produces a single quantitative score for each concept, which ensures appropriate for comparing concepts.

4.2 Concept A – Structured operator Workflow with Guided Teach in programming

4.2.1 Methodological Framework

Concept A retains the conventional teach pendant paradigm as the mechanism for defining robot motion and introduces a software-based layer that structures and sequences the operator interaction. In this method, the operator is guided through a step-by-step workflow that collects process inputs, including the desires bending sequences, bend side selection and current workpiece positions. The operator then executes each action directly via robot teach pendant or control interface using the function blocks.

This Concept is grounded in the established practice of teach pendant programming described in section2.1. As identified in the literature, teach pendant programming is reliable and widely used in structured manufacturing environment where part geometries and process parameters are fixed [1]. The guidance layer introduced in this concept (A) reduced the risk of operator errors and provides a structured interface than unguided teach pendant operation, but it does not alter the fundamental that all robot position and waypoints are defined through direct physical interaction with the robot.

Regripping operation, orientation changes, full side bending operation for a rectangular sheet under this concept A remains the responsibility of the user to define and execute. The system provides instructional prompts but does not execute, validate or generate any waypoints and position data. No interface to the robot runtime environment is required beyond standard teach pendant connectivity, so the concept does not operate within the drag and bot virtual simulation runtime and does not satisfy SR-01 or SR-02 as defined in Section 3.2.4. The system does not possess object tracking subsystems, evaluate geometric constraint, grip position calculations or assess

kinematic feasibility before execution.

4.2.2 System Structure

The system structure of concept A consists of two layers. A dialogue management layer runs on an external computer and presents the operator prompt through functional blocks or graphical interface by manual program generation. A robot controller layer receives waypoint generated manually by the operator through teach pendant and execute them. No intermediate computation layer for geometric reasoning, constraint evaluation, or motion generation exists between these layers. The architecture imposes no dependency on object tracking sensor data or trajectory generation interfaces.

4.2.3 Critical Evaluation

Concept A straightforward to implement and compatible with any industrial robot controller without software modification. The operation of the system is fully transparent and observable by the operator, which reduces the risk of unreachable and kinematics error occur during the unreachable target pose. These characteristics make it suitable for low variety, structured production scenario where process parameters are stable across batches.

However , the fundamentals limitation if this concept are significant in the context of research gap ,programming effort is not yet reduced because knowledge of robot kinematics , tooling constraints , regripping strategies and bending sequence algorithm should need to applied by the operator for every session [1], [3]. Minor changes in sheet geometry , workpiece holder placement requires full re teaching of the affected trajectories , which contradicts the flexible objective stated in section 3.1 [1], REF 26. Regripping operation and full side bending operations must ne manually planned , which introduces a chances of operator error and increase the cycle time .No pre execution validation of target poses is performed , kinematic configuration are discovered only during the physical or virtual robot motion which leads to safety risk and a source of unplanned downtime [1].

The software guidance layer of concept A provides process management but no automation level, it addresses the unstructured operator workflow without addressing root cause, which is the requirement for manual position definition. Concept A does not contribute to the primary objective of automatic program generation and failed to satisfy most of the functional and non-function requirements defined in 3.1. and REF NFR.

4.3 Concept B – Automatic Program Generation based on rules with Integrated Constraint Validation.

4.3.1 Methodological Framework

Unlike Concept A, which retains manual position definition, Concept B eliminates it entirely. Concept B proposes an automation level, rule based, automated program for generation system in which all robot waypoints, grip positions, regrip sequences, opposite side handling operations and strategies are derives automatically from geometric input parameters and a structures YAML configuration file: operator interaction is limited selection of required output selection before execution and confirmation of generated programs visually. No manual waypoint generation, direct teaching or any runtime operation or decisions are required during program execution.

The system communicates with the robot runtime through AnyRobot interface, a robotics application program (API) Developed by KEBA \ Drag and Bot GmbH that provides standardized method for controlling industrial robots from different manufacturers via WebSocket communication[24]. The planning Software operates as the WebSocket client, the drag and bot runtime act as the server. This architecture establishes a clean separation between the planning computation layer and robot execution layer, which is a recognized design principle [25] .

Workpiece pose data is retrieved in real time from drag and bot object tracking subsystem and processes by an orientation validation component , which determines whether the tracked sheet orientation is admissible for the planned bending operation by applying a three level priority strategy ,a side normalized orientation is evaluation first , followed by original tracked orientations by the object tracker and finally a 180 degree rotated strategy , when neither of the first two falls within the configurable window. This Three level hierarchy ensures that valid gripping orientations and positions are identified for any initial sheet placement on the cell floor, which eliminates the need for operator prepositioning. Before grip position calculation, the program validated the sheet position is within the robot cell. This is applied to avoid the kinematic reachability error occurred during the program execution. The zone-specific admissibility limits are specific limits are robot cells configuration are configured in the input sections and require no code modification when cell layout changes. This design decision satisfies NFR 02as defined in table 3.2 and NFR 05 as defined in table 3.2.

Grip position are computed by a parameterized grip planning component s that uses the sheet dimension, mass suction cup diameter , rated shear

force and edge clearance margin which all are defined in the input configuration files :candidate grip position reevaluated against edge clearance and cup spacing constraints and against the vacuum holding force which is derived from the relationship between suction pressure , cup contact areas and dynamic loading during transport [12].Grip feasibility is evaluated before kinematic reachability assessment, ensuring that only variable grip configurations are submitted to the motion generation layer. This ordering prevents unsafe handling states that would result from a grip position that is kinematically reachable but physically unable to reach the sheet during transport [12] .

When the current grip configuration does not allow the targeted pose to be reached, an automated regrip cycle is initiated. The cycle places the sheet on a designated orientation table, repositions the robot to reset and then regrasp the sheet at an orientation that restores full reachability. This regrips logic directly satisfies FR03 as defined in table 3.1 .

Opposite side bending is also supported through this concept through a dedicated module that executes a structured sheet flip sequence. After all primary bending operations are completed, the program evaluates whether the operated has selected for opposite side bends. If so, the sheet passes to flip position through a coordinated sequence of motion, gripper activation, and object attachment operations targeting positions to flip the sheet. Afte the flip, the opposite bending sequence is executed under the same validation, including orientation normalization, grip feasibility evaluation and reachability assessment before the waypoints is generated. This enforcement of validation across both primary and opposite side bending operation ensures consistent safety behavior throughout th full step bending process [7]

The motion trajectories are generated in linear cartesian motion commands using minimum jerk velocity interpolation profile REF 16: This profile minimizes the third derivatives of position over the motion, producing smooth velocity transition between waypoint that reduce peak acceleration and gripper loading forces during sheet transport. This is requirement for maintaining vacuum grip stability under dynamic conditions [12].A singularity avoidance strategy ensures the robot arm does not pass through kinematically degenerate configurations during the approach phase [1]. All motion targets are expressed relative to the pre-defined tool center point registered in the drag and bot tool management system. This reference is mandatory because SR-03 requires that all generated flange poses be consistent with the physical gripper geometry of the KEBA press brake cell, and the defined

TCP entry encodes that geometry with the exact pose values specified in Section 3.2.4 [24].

4.3.2 system structure

The structure of Concept B consists of Five layers as defined in section System-Architecture-and-Design. The input and configuration layer that collects all required inputs for the programming. The geometry and constraints evaluation layer applies grip feasibility rules, edge clearance checks and zone-based orientation constraints. The planning and decision logic layer determines the execution strategy, which decides the program to be direct execution or regrip required execution or workpiece flipping strategy, based on reachability and orientation validity . The trajectory planning layer produces linear cartesian waypoints with minimum jerk velocity profiles. The safety and validation layer applies pre-execution pose check before the command is published to the robot runtime.

This WebSocket channel of the Any robot interface allows to connect all five layer to drag and bot virtual simulation run time [24] .The configuration-driven approach allows all process parameters to be trimmed from program logic, allowing the solution to tackle new workpiece geometries simply by modifying the content of configuration files. This was in direct satisfaction of non-functional requirement NFR-02 as described in Section 3.2.2.

4.3.3 Critical evaluation

Concept B is aligned with the research gap stated in Section 2.5 for every dimension identified in the literature. Define waypoints, plan regrips, and sequence work on the opposite side automatically from geometric parameters instead. A pre-execution pose validator guards against unsafe execution states before motion starts. This allows for configuration driven architecture that can adapt to new part geometries without the need for source code changessatisfying FR-01, NFR-03, NFR-04, and NFR-05. Configuration-driven parameterization satisfies NFR-02. The system operates exclusively within the drag and bot runtime via the AnyRobot Interface, satisfying SR-01 and SR-02. All motion targets reference the pre-defined tool center point, satisfying SR-03. All fifteen requirements in Chapter 3 are therefore addressed by the concept design.

The primary constraints that Concept B operates within are those identified within Section 3.2.3. The system operates within a rectangular sheet geometry with fixed 90-degree bend angle constraints. The integration with

the KEBA robot controller hardware, which is part of the KEBA robot driver layer within the drag and bot software stack [24], is identified within the future work section and is not included within the scope of this thesis. All validation has been conducted within the drag and bot virtual simulation environment, which is discussed in this section. The virtual simulation environment used for this evaluation is illustrated in figure 4.1 .The environment represents a simulated press brake cell configured with the drag and bot platform , including the robot , workpiece and interactions required for the program execution .These constraints are explicit and not indicative of design flaws but, rather, define the scope within which the design concept operates.

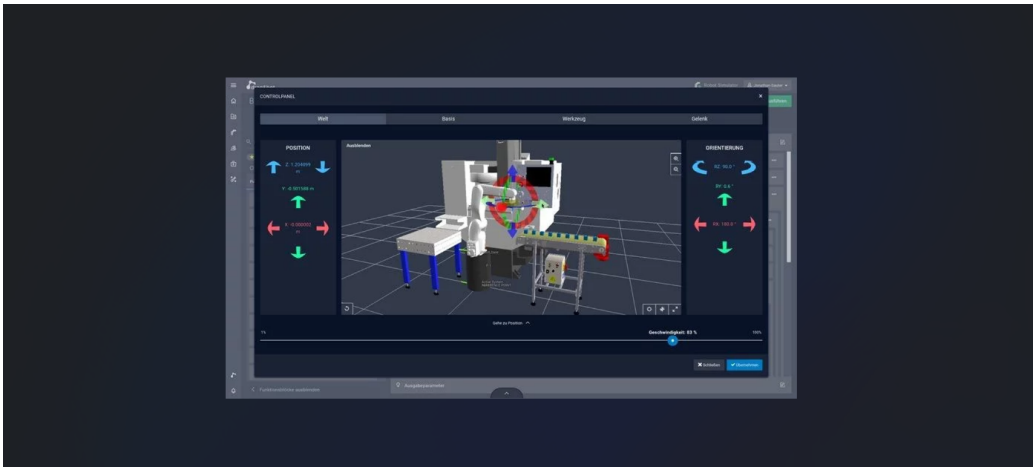


Figure 4.1: Virtual robotic environment used to evaluate the proposed concept (Drag & Bot platform).

4.4 Concept C – CAD Integrated Offline Programming with Digital Twin Simulation

4.4.1 Methodological Framework

Concept C proposes a computer aided design (CAD) integrated offline programming (OLP) approach in which a digital model of the sheet metal workpieces and the robot cell used to plan , generate and validate robot program in a simulation environment , without using physical robot interaction during the programming .The OLP software evaluates bending sequences , grip positions and regrip programming, kinematic reachability and genreted robot program based on the CAD geometry of the workpiece and tooling .The generated programs are validated inside a simulation before transfer to the robot controller for execution.

This approach corresponds to the offline programming section 2.2. Industrial Olp tools such as ABB Robot Studio ,KUKA: sim And FANUC ROBOGUIDE implement this approach and are used in industry for robot program generation in structured production environment [3] .The digital twin based evaluation and validation reduces the occurrence of risk by identifying unreachable poses and collision condition before real execution, which has been identified as significant advantage over the teach pendant programming. [27]

4.4.2 System Structure

This concept consist of four layers .A cad modelling layer provides digital representations of the robot , press brake , tools and workpiece .An OLP and simulation layer uses the URDF(Unified Robot Description Format) representations to plan the algorithm for robot program through built in kinematic solver .A post processing layer converts the generated programs into controller specific syntax's robot controller layer communicates the program for exception. Accurate digital twin representation of all cell components are required at the modelling layer small errors in modelling the digital twin model propagate as execution error that require manual re programming during commissioning.[1] [3].

4.4.3 Critical Evaluation

Concept C offers meaningful automation advantages for structured, high-volume production environments where accurate cell models are available and OLP tool investment is justified. Simulation-based validation provides strong pre-execution assurance, and the approach is supported by mature commercial software [1][27]. These characteristics make it the dominant paradigm in industrial robotic bending automation at the time of writing.

For the scope of this thesis, Concept C presents three fundamental disadvantages that result in its rejection. First, the implementation requires accurate CAD models of the complete robot cells, including the robot, press brake, tooling, fixtures, and workpieces — which represents a substantial engineering preparation effort that is not available within the thesis timeframe.

Second, commercial OLP systems involve high license fees while demanding expert-level instruction - resources simply unavailable within this project's constraints REF Engineering Design: A Systematic Approach . Besides cost, mainstream OLP platforms lack native compatibility with both the drag and bot simulation framework and the AnyRobot WebSocket

system, which define the core setup targeted here [24]. Connecting such a tool would mean building a bespoke bridge to the drag and bot engine - an extra burden unjustified by timeline or ambition.. Even if the first two limitations were resolved, this third limitation would disqualify Concept C, because SR-01 and SR-02 are non-negotiable constraints from the industrial partner environment.

4.5 Structural Comparison of Concepts

Table 4.1: Structural comparison of Concept A, Concept B, and Concept C across key design features

Design Feature	Concept A	Concept B	Concept C
Waypoint Generation Method	Manual operator teach-in for every new part geometry [1]	Automatic derivation from sheet geometry and YAML configuration parameters [24]	CAD-based OLP solver algorithms; new CAD model required per variant [3]
Regrip Automation	No automation. Operator plans and executes each regrip sequence manually [2]	Full automation via dedicated regrip module: IO control, attach/detach, and rotation sequencing under YAML-configurable parameters [2][7]	Regrip positions defined in CAD simulation; execution not automated at runtime [27]
Opposite-Side Bending	Manual sheet flip by operator; no software support	Dedicated flip module executes structured sheet transfer via predefined cell objects; same validation hierarchy as primary bending [7]	CAD simulation resolves flip; runtime execution remains manual [27]
Pre-Execution Pose Validation	None. Unreachable poses discovered at runtime during physical robot motion [1]	Orientation validation evaluates admissibility using a three-level priority strategy	Orientation validation performed in CAD simulation; no runtime checks [3]
Configuration Adaptability	Not supported. Full re-teaching required per geometry variant [3]	sheet dimensions, bend sides, and grip parameters configurable without code modification	CAD-based approach requires new model and re-simulation for each variant [27]
Platform Compatibility	No dependency on drag and bot or AnyRobot Interface; operates via teach pendant only. Does not satisfy SR-01 or SR-02	ROS-based interface with drag&bot AnyRobot middleware for real-time command execution [24]	Incompatible: standard OLP tools do not support direct integration with the drag and bot runtime or AnyRobot Interface; violates SR-01 and SR-02
Tool Centre Point Definition	TCP not software-defined; operator sets tool geometry manually on teach pendant. SR-03 (project_3_tcp) cannot be enforced	All motion targets computed relative to the project_3_tcp TCP registered in the drag and bot tool management system, satisfying SR-03	TCP defined within OLP CAD model; independent of the project_3_tcp constraint required by SR-03
Implementation Scope Feasibility	High feasibility but provides no automation contribution to thesis objective	Fully implementable in Python within dragand bot virtual simulation; achievable within thesis scope [24]	Requires commercial OLP software, accurate cell CAD models, and custom integration; exceeds thesis scope [1]
Research Gap Addressed	No automation; manual processes dominate	Automated waypoint generation, regrip sequencing, and flip execution with real-time validation; addresses key automation gaps in current state of the art [7]	Partial - Focuses on CAD-based simulation and OLP without runtime automation; does not address real-time adaptability

Before applying the quantitative WSM, the three concepts are compared

at the structural level to make the qualitative differences explicit and to verify that the comparison covers all requirement categories. This structural comparison explicitly includes SR-01 and SR-02, because platform compatibility is a binary disqualifier rather than a continuous criterion - a concept that cannot operate within the required platform fails the evaluation regardless of its scores on other dimensions. Table 4.1 presents this comparison across nine features. Table 4.1 makes the selection case for Concept B unambiguous at the structural level. Concept A achieves a positive rating only on implementation feasibility, because its teach-pendant architecture requires no additional tooling - but this advantage is outweighed by its failure on all five automation-related features and both platform requirements SR-01 and SR-02. Concept C achieves partial positives on waypoint generation and pre-execution validation through its CAD-based approach, but these are negated by the SR-01 and SR-02 incompatibility, which is a structural disqualifier rather than a weighted disadvantage. These structural observations confirm in advance the numerical outcome of the WSM presented in Section 4.6

4.6 Concept Decision-Weighed Sum Model (WSM)

4.6.1 Methodology

The quantitative concept selection technique used is the Weighted Sum Model. Each evaluation criterion in the WSM is given a weight based on how important it is to the design objectives. Each concept is then scored against each criterion on a scale of 1 to 5, with 5 denoting full satisfaction and 1 denoting non-fulfillment. Each concept's overall weighted score is calculated by multiplying the corresponding score by each criterion weight. The idea that receives the highest total weighted score is chosen.

4.6.2 Evaluation Criteria and Weights

Six evaluation criteria are defined directly from the functional, non-functional, and software platform requirements of Chapter 3. The weight assigned to each criterion reflects its relative contribution to resolving the research gap identified in Section 2.5, as justified below

Programming Effort Reduction (25%) -This criterion carries the highest weightage because it directly addresses the primary research gap. This

makes automation easier by reducing a lot of manual programming especially when parts change frequently. This gap is consistently identified in literature as the most significant barrier to increased automation in robotic bending cells.

Automated Regrip and Opposite-Side or sheet flipping Support (20%) : According to Section 2.4, the most difficult and time-consuming parts of robotic press brake bending that are not sufficiently handled by current methods are regripping and opposite-side handling[2][7]. The 20% weight reflects that automating these operations is a core deliverable, not a secondary feature, of the research gap resolution per FR-03 and FR-05 in Table 3.1.

Pre-Execution Pose Validation (20%) : Before robot motion is started, infeasible poses must be identified in accordance with safety regulations NFR-03, NFR-04, and NFR-05. In robotic bending cells, executing unreachable target poses is a major source of system failure as well as a safety risk.

Flexibility for Changing Part Geometries (15%) : One of the main goals derived from the research gap is the capacity to adjust to new workpiece variations without reprogramming. NFR-02 is directly satisfied by configuration-driven adaptability, and the 15% weight indicates that this is a secondary goal following the primary automation requirements.

Platform Compatibility drag and bot Runtime and AnyRobot Interface — SR-01, SR-02 (10%) :The industrial partner environment mandates the drag and bot virtual simulation runtime and the AnyRobot Interface as the exclusive execution and communication platform, formalised as SR-01 and SR-02 in Section 3.2.4. A concept that fails SR-01 or SR-02 cannot be implemented or validated within the thesis scope regardless of its theoretical merits, which is why a score of 1 on this criterion effectively disqualifies a concept. SR-03 (project_3_tcp) applies equally to all platform-compatible concepts and therefore does not independently differentiate them; it is reflected in the Concept B system structure in Section 4.3.2.

Operator Usability (10%) : Non-functional requirement NFR-01 says that the system must be usable by non-expert operators with little interaction during normal operation. The 10% weight shows that usability is an

important quality attribute, but it is not as important as the main automation requirements.

4.7 Result of the weighted Sum Model

Table 4.2 presents the complete WSM evaluation with score justifications and weighted totals for all three concepts

Table 4.2: Structural comparison of Concept A, Concept B, and Concept C across key design features

Evaluation Criterion	Wt.	Concept A — Justification	Sc.	Concept B — Justification	Sc.	Concept C — Justification	Sc.
Programming Effort Reduction	25%	All waypoints defined manually per session; no reduction in programming effort	1	Waypoints derived automatically from geometric parameters; no manual teaching required	5	CAD solver reduces teaching effort but requires model setup per variant	3
Automated Regrip and Opposite-Side Support	20%	No regrip logic; operator plans and executes all regrip and flip operations manually	1	Full automation via regrips and opposite-side modules with IO and attach control	5	Regrip targets defined in simulation; execution not automated at runtime	2
Pre-Execution Pose Validation	20%	No validation; unreachable poses detected only during physical robot motion	1	Orientation validation and spatial zone classification executed before motion generation	5	Simulation validates program before export; no runtime check available	3
Configuration Adaptability	15%	Full re-teaching required per geometry variant; scales poorly	1	Sheet parameters stored in YAML; new geometry requires only configuration update	5	New CAD model and re-simulation required for each variant	3
Platform Compatibility	10%	No dependency on drag&bot or AnyRobot Interface; incompatible with the required industrial partner development platform. SR-01 and SR-02 not satisfied	4	Fully implementable in Python within drag&bot virtual simulation; achievable within thesis scope	5	Standard OLP tools cannot integrate with drag&bot runtime or AnyRobot WebSocket Interface; violates SR-01 and SR-02	2
Operator Usability	10%	Expert robot programming knowledge required for each session; high cognitive load	2	Operator selects bend side and confirms execution; decisions handled by the system	5	CAD interface reduces robot expertise requirement; specialist CAD knowledge required	3
Weighted Total Score	**100%**	Concept A	**1.40**	Concept B — SELECTED ✓	**5.00**	Concept C	**2.70**

4.8 Analysis of results and Concept Decision

The WSM produces total weighted scores of 1.40 for Concept A, 5.00 for Concept B, and 2.70 for Concept C. Concept B achieves the maximum possible score and is therefore selected as the concept to be developed and implemented in Chapter 5. The margin between Concept B and the nearest

alternative is 2.70 points, which is not a marginal result: it reflects systematic superiority across every criterion, not a single dominant factor.

Concept B meets all six requirements to the highest degree, as indicated by its score of 5.00. Because waypoints are automatically derived from geometric parameters and there is no manual teaching step, programming effort is eliminated. Because specialized software modules implement gripper IO control, object attach and detach sequencing, and multi-step rotation under YAML-configurable parameters with all behavior fully specified before runtime, regrip and opposite-side bending achieve full automation. Orientation validation and spatial zone classification are two independent components that structurally guarantee pre-execution validation because they are both called before any motion command is generated, preventing an invalid pose from reaching the motion generator. NFR-02 is satisfied by configuration-driven sheet parameterization. As required by the industrial partner, the system satisfies SR-01 and SR-02 by operating solely within the drag and bot runtime via the AnyRobot Interface. SR-03 is satisfied since all motion targets make reference to the project_3_tcp tool center point. Within the parameters of the thesis, the Python-based implementation is completely feasible [17]. By restricting interaction to necessary pre-execution choices, the dialog-based operator interface complies with NFR-01.

Because Concept A does not meet the five most important requirements, it receives a score of 1.10. Regardless of any other characteristics, its platform compatibility score of 1 indicates that it is incompatible with the industrial partner development platform since it does not function within the drag and bot runtime, meeting neither SR-01 nor SR-02. The concepts teach-pendant architecture prevents it from contributing to any aspect of the research objective, as evidenced by the scores of 1 on programming effort, regrip automation, pose validation, and configuration adaptability [17].

Because Concept C's CAD-based OLP approach tackles waypoint generation, pre-execution validation, and configuration adaptability during the simulation phase, it receives a score of 2.70. However, the CAD approach's lack of runtime constraint evaluation and requirement for per-variant model rebuild rather than configuration file update limit the score. The platform compatibility score of 1 indicates a structural disqualifier: standard OLP tools are unable to integrate with the AnyRobot WebSocket Interface or the drag and bot runtime, which violates SR-01 and SR-02. This platform incompatibility alone would prevent Concept C from being implemented or validated within the thesis environment, even putting aside the resource and cost constraints [1] [17].

Three conceptually different methods for automatic robot program generation in press brake sheet metal bending were developed and assessed in this chapter. The evaluation adhered systematic engineering design methodology because it offers an organized framework for evaluating options based on predetermined criteria prior to committing to implementation, which is necessary for a thesis-level concept decision [17]. All three of Chapter 3's requirement categories—functional requirements FR-01 to FR-07 (Table3.1), non-functional requirements NFR-01 to NFR-05 (Table 3.2), and software platform requirements SR-01 to SR-03 (Table 3.3, Section 3.2.4)—were used to develop the evaluation criteria.

Because Concept A operates independently of the drag and bot platform and AnyRobot Interface required by the industrial partner environment, its teach-pendant architecture cannot reduce programming effort, provide automated constraint validation or regrip support, or satisfy SR-01 or SR-02. As a result, the guided teach-in approach was rejected. These failures are structural: according to Table 3.1, FR-01, FR-03, FR-05, and FR-06 require automated geometric reasoning, which the two-layer teach-pendant architecture is unable to supply.

Concept C,method of programming offline: Integration with CAD software for creating digital cell models for use in offline programming (OLP). This method was disregarded because required commercial OLP tools do not exist; accurate digital cells are also not included in the thesis scope. Therefore, the approach as a practical limitation has a structural constraint due to its incompatibility with drag and bot's run-time and the AnyRobot Interface (Section 3.2.4, SR-01, SR-02) [1][3]. Consequently, any partial automation benefits associated with this method could not be achieved using simulation techniques due to these constraints also being exhibited in the simulation environment within this thesis.

The rule-based automatic program generation system of Concept B was selected because it achieves the maximum weighted score of 5.00 in the WSM, satisfying all fifteen requirements of Chapter 3. This automated system eliminates programming time through automatic determination of waypoints, automates regrip and opposite side bend sequences through dedicated software modules, and verifies whether there is an admissible orientation and spatial reachability prior to the generation of any motion; (b) operates solely in the drag and bot virtual simulation runtime, through use of the authenticated AnyRobot WebSocket Interface. This satisfies both SR-01 and SR-02. All motion targets are computed with respect to the project_3_tcp (tool centre point); therefore, it meets requirement SR-03. The technical de-

tails about Concept B, including the entire software architecture, interface specifications, algorithm derivation and risk analysis can be found in Chapter 5.

5 Implementation

5.1 Introduction and scope

The development of product phase transforms the selected concept into a fully realized technical system. Concept B, chosen through the weighted sum model with a total weighted score of 5.00. Exceeding Concept A at 1.60 and concept C at 2.70 is implemented here as an executable software architecture that generate robot programs automatically for press brake sheet bending. Concept B was the only concept which satisfies most of the requirements as defined in chapter [3]. The implementation addresses four accountability domains. The first is the system architecture, covering the layered module structure and rationale for every structural decision. The second is the interface specification, describing all data exchanges between modules and between planning software and drag and bot runtime. The third is the design failure mode and effect analysis in section, which links each failure mode to a risk priority number and countermeasure, fulfilling the peruse risk assessment obligation. The fourth is the software implementation record, presenting algorithmic derivation and configuration structures so that every design is traceable to a requirement. The system operates within the drag and bot virtual simulation environmental communication with the robot runtime is conducted through the Any Robot Interface over a persistent authenticated WebSocket Channel, satisfying all the software requirements' process parameters are loaded from a single input configuration file at the staring , satisfying NFR-02 by enabling workpiece changeover through a file edit alone .Operator input is provided through a purpose built interactive 3D sheet bending viewer embedded in the drag and bot operator cockpit , which give a visual geometry feedback reduces the risk of incorrect bend side selection . Ubuntu Linux is required as the development platform because drag and bot is built on ROS, which has no production release for Windows or macOS [28] Fig 5.1 shows the complete bending cycle execution from session start through workpiece gripping. Each stage maps directly to a section of this chapter. The chapter is organized as follows, system architecture, module overview, communication stack configuration architecture and operator interface are presented first. The simulation Environment and spatial zone classification flow. Each functional subsystem is then developed in the execution of a bending session, covering pre-execution safety valida-

tion, workpiece orientation strategy, gripper control, grip position planning, regrip sequencing, sheet flipping and bending, trajectory generation and post bend unloading. A design Failure mode and effect analysis concludes the implementation record and the final section maps all the requirements from chapter 3 to their implemented subsystems.

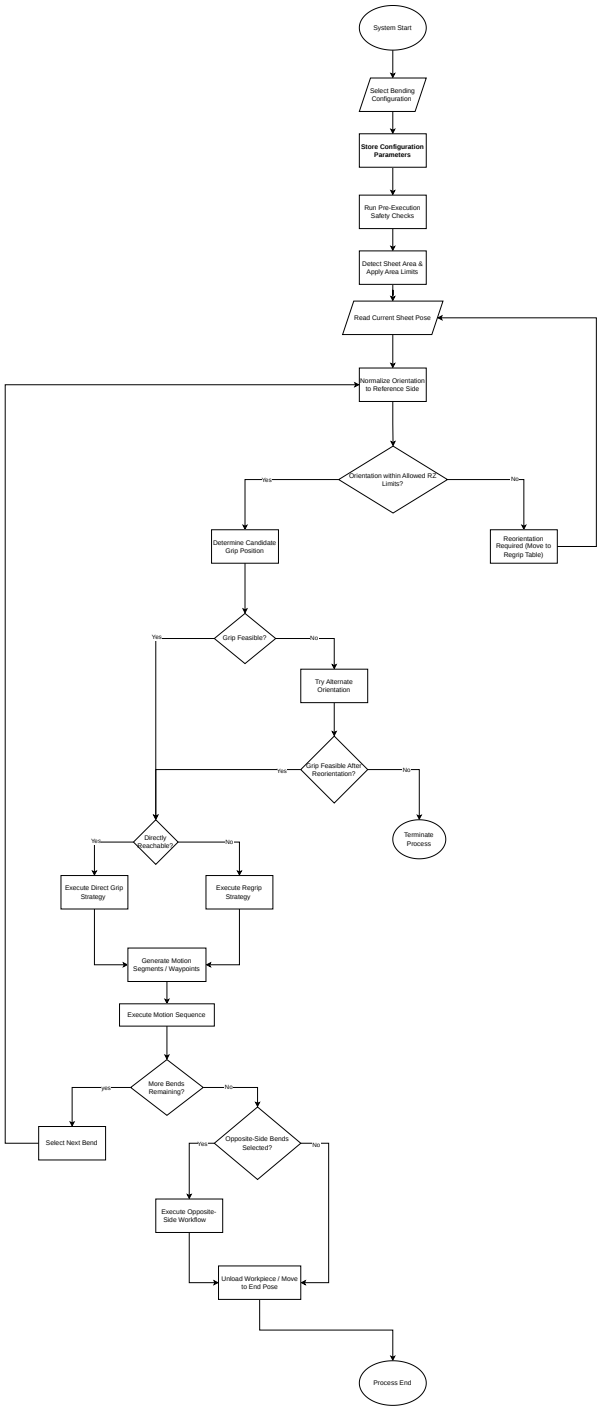


Figure 5.1: System architecture

5.2 System architecture

5.2.1 Layered Architecture Design and data flow

The system adopts a layered architecture in which each module depends exclusively on module in the layered directly beneath it. This pattern is selected because the press brake automation context imposes three constraints that unstructured flat module design cannot satisfy simultaneals, independently testable constraints evaluation, maintenance safe replicability and structural enforced safety ordering.

Testability is achieved by enabling the module to be evaluated using synthetic pose data and YAML-based configuration inputs, without the need for an active WebSocket connection. This isolation is essential for verifying the orientational classification mathematics and grip feasibility calculations, because that computation cannot be inferred correctly from a live system alone. Replaceability is ensured by a modular architectural approach in which changes to the grip position algorithm are localized within the constraint evaluation layer. The planning and motion layers are structurally isolated and maintain no direct dependencies on constraint evaluation modules, thereby satisfying the requirements of NFR-02. Safety ordering is maintained through a structured design in which pre-execution safety validation is implemented in an independent layer executed prior to any motion commands. This approach ensures that future modifications cannot inadvertently reposition safety checks after motion initiation, thereby satisfying FR-07 and NFR-03.

Five distinct layers are defined within the system architecture, each with a clearly assigned responsibility. Layer 1 (Input and Configuration) is responsible for acquiring all required parameters from the input file, along with the operator's bend-side selection, prior to the establishment of any WebSocket connection. Layer 2 (Geometry and Constraint Evaluation) performs the computation of geometric and physical constraints based on live tracker data, while remaining strictly limited to evaluation tasks and producing no motion commands. Layer 3 (Planning and Decision Logic) determines the overall execution strategy and coordinates the interaction between Layers 2 and 4, without directly generating motion instructions. Layer 4 (Motion Generation and Communication) serves as the exclusive interface for robot communication, generating LIN and PTP waypoints, converting them into JSON format, and transmitting them via an authenticated WebSocket connection. Layer 5 (Safety and Validation) is invoked at the beginning of every session and enforces a mandatory validation step, preventing further execution

until explicit operator confirmation is obtained. The complete architectural structure is illustrated in Figure

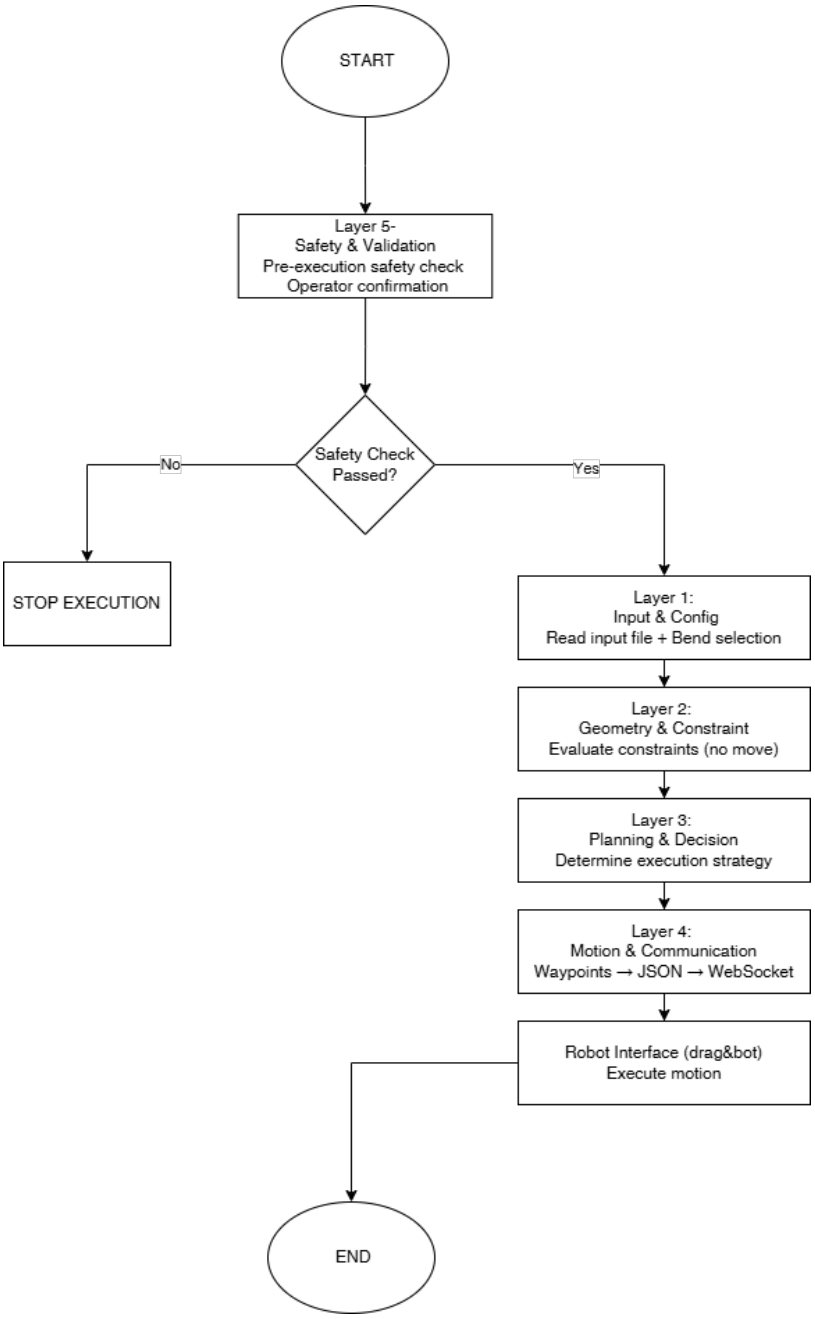


Figure 5.2: Five layer architecture

The progression of data throughout a bending cycle follows a structured transformation across the system layers. At the initial stage, sheet geometry, bend-side selection, and TCP parameters are provided as input. These inputs are subsequently interpreted through geometric and constraint-based evaluation, where raw tracker pose data is converted into a validated pickup orientation and a feasible grip position. Based on this processed information, an appropriate execution strategy is established before being translated into

motion commands, which are encoded in LIN-based JSON format and transmitted to the runtime environment.

The reliability of the overall process depends on a small number of critical transformation stages. Any inaccuracy introduced at these points propagates forward and affects all subsequent computations and decisions. In particular, the correct determination of pickup orientation is essential for successful workpiece acquisition. Likewise, the evaluation of grip feasibility ensures stable handling under dynamic conditions and prevents slippage during motion. During trajectory generation, a balance must be maintained between path fidelity and the limited processing capacity of the robot controller. This requires a controlled reduction of trajectory points while preserving the essential characteristics of the intended motion.

5.2.2 Module Overview and dependency Discipline

The implementation consists of software modules organized across the five system layers. Table 5.1 summarizes each module in terms of its layer assignment, functional role, and primary responsibility. Each module is defined with a single primary responsibility to ensure clear fault attribution during failure analysis. Assigning multiple responsibilities to a single module would make it difficult to associate a specific failure mode with a well-defined functional unit.

Table 5.1: Software modules across system layers and their primary responsibilities

No.	Layer	Module	Primary Responsibility
1	L1	Session launcher	Coordinates YAML loading, 3D viewer initialisation, and session start-up before runtime connection.
2	L1	Configuration reader	Loads and validates configuration data and provides a consistent parameter source for the remaining modules.
3	L5	Safety checker	Enforces operator confirmation and safety validation before execution begins.
4	L2	Zone classifier	Classifies tracker data into spatial zones and determines admissible orientation strategies.
5	L2	Orientation selector	Determines the pickup orientation on the basis of geometric evaluation of the workpiece.

No.	Layer	Module	Primary Responsibility
6	L2	Face orientation converter	Converts tracker pose data into a reference-face representation using rotation transformations.
7	L2	Grip position planner	Computes feasible grip positions while considering edge clearance and holding constraints.
8	L2	Pickup orientation validator	Verifies tool alignment and applies corrective transformations where required.
9	L4	Trajectory generator	Generates waypoint sequences with path refinement, sampling control, and singularity avoidance.
10	L4	Regrip motion executor	Executes regrip operations, including transport, release, and re-acquisition.
11	L3	Primary bending orchestrator	Coordinates execution flow across orientation, planning, and motion modules.
12	L3	Regrip sequencer	Determines transition sequences between grip configurations.
13	L3	Opposite-side runner	Evaluates and executes alternative orientation strategies when required.
14	L3	Multi-stage regrip loop	Handles iterative repositioning for complex bending sequences.
15	L3	Post-bend unload sequencer	Controls unloading of completed workpieces to the designated buffer location.
16	L4	Waypoint data structure	Encapsulates robot motion parameters, including pose, velocity, and blending information.
17	L4	Digital IO handler	Manages vacuum control and safety-related input/output signals.
18	L4	WebSocket client	Handles authentication, session management, and message transmission to the runtime interface.
19	L4	Command type definitions	Defines motion categories, coordinate conventions, and JSON message formats.

Three structural properties ensure that the architectural constraints are consistently enforced across the system.First, the WebSocket client and the command type definitions together constitute the exclusive communication layer. No mod-

ule outside Layer-4 constructs WebSocket messages or performs direct network operations. As a result, authentication handling and connection recovery are implemented centrally and remain consistent across all runtime interactions. Second, the safety checker is positioned within Layer 5, which is not imported by any lower layer. This separation prevents planning or execution modules from invoking safety validation during runtime. Instead, safety checks are enforced strictly at the beginning of a session, ensuring that they cannot be bypassed or reordered during execution. Third, the trajectory generator serves as the sole interface for dispatching motion commands to the runtime. All waypoint sequences are routed through this module, ensuring that workspace clamping and singularity checks are consistently applied before execution. This guarantees that all generated motions comply with the defined safety and feasibility constraints.

5.2.3 Virtaul Enviornment setup

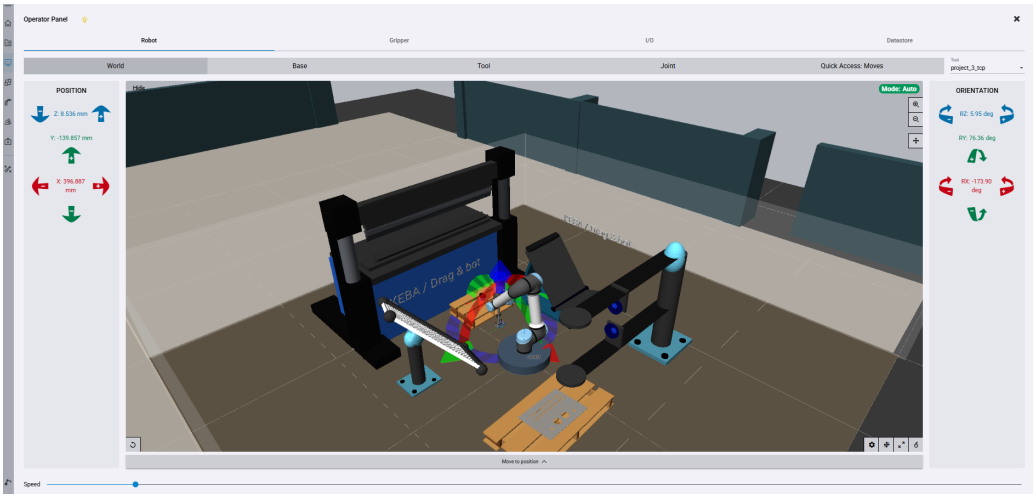


Figure 5.3: Virtual robotic environment used to evaluate the proposed concept within a simulated press brake cell (Drag & Bot platform).

ADD FIGURE BLOCK DIAGRAM AND EXPLAIN THE SIMULATION ENVIRONMENT, ROBOT MODEL AND TCP CONFIGURATION, SPATIAL ZONE CLASSIFICATION

5.2.4 Communication Stack, Authentication interface Specification

All runtime communication passes through the AnyRobot Interface, selected because it is the only supported external API for drag and bot. Direct ROS client libraries are excluded because they would couple the planning software to a specific ROS version, violating NFR-02 [24] [28]. The AnyRobot Interface translates JSON over a persistent WebSocket channel into native ROS operations using the RosBridge v2 protocol, enabling the entire planning software to run in standard Python 3 without a ROS installation on the planning machine.

WebSocket (RFC 6455) is chosen over HTTP because it supports a persistent connection that allows continuous data transfer. In contrast, HTTP requires a new TCP connection for each request, which introduces a delay of approximately 3–15 ms per batch. When sending a trajectory of 400 points in batches of 50 commands, these delays accumulate to 24–120 ms. This added delay can cause the robot controller's command buffer to run empty, leading to interruptions during motion execution. By maintaining a continuous connection, WebSocket avoids this overhead and ensures smooth and uninterrupted command transmission.

Figure 5.4 presents an overview of the system communication architecture. The planning software establishes a persistent WebSocket connection to the robot controller via a defined network endpoint (e.g., `ws://192.168.x.x:9091`). Communication is organized through a set of dedicated interfaces, each responsible for a specific function within the system. The Ros Bridge v2 interface is used to transmit motion commands, while the `rosauth` service ensures secure authentication using the SHA-512 hashing algorithm. This method encodes authentication data into a non-reversible format, preventing the transmission of sensitive information in plain text. Motion trajectories are transmitted in batches via the `/command_list` interface, and real-time object information is retrieved through `/dnb_object_manager/list_objects`. Gripper control is performed using `/robot/set_digital_ios`, whereas object attachment and detachment are handled through the corresponding manager interfaces. User interactions, such as confirmations and prompts, are managed via the `human_interaction/request_hi` interface. The YAML configuration is not part of the runtime communication. It is loaded once during system initialization and shared across the relevant modules. This approach ensures that missing or incorrect parameters are detected early, before any communication with the robot is initiated [29].

On connection establishment, the WebSocket client performs the rosauth SHA-512 authentication handshake, because the drag and bot WebSocket port is accessible to any process on the local network and an unauthenticated endpoint would allow arbitrary motion command injection. The MAC is computed by hashing with SHA-512 concatenation of the shared runtime secret, the interface identifier, destination, a fixed nonce, session timestamps, and the access level string.

The authentication token is computed based on the procedure defined in the drag&bot AnyRobot Interface specification [24]:

$$\text{MAC} = \text{SHA-512}(\text{secret} \parallel \text{id_string} \parallel \text{dest} \parallel \text{nonce} \parallel \text{t} \parallel \text{level}) \tag{5.1}$$

SHA-512 is selected due to its strong cryptographic properties, as its 512-bit output makes pre-image and collision attacks computationally infeasible within the operational context of this application. The shared secret is stored as an environment variable to prevent exposure to version control systems. The concatenation order is strictly defined by the AnyRobot interface specification and must be followed

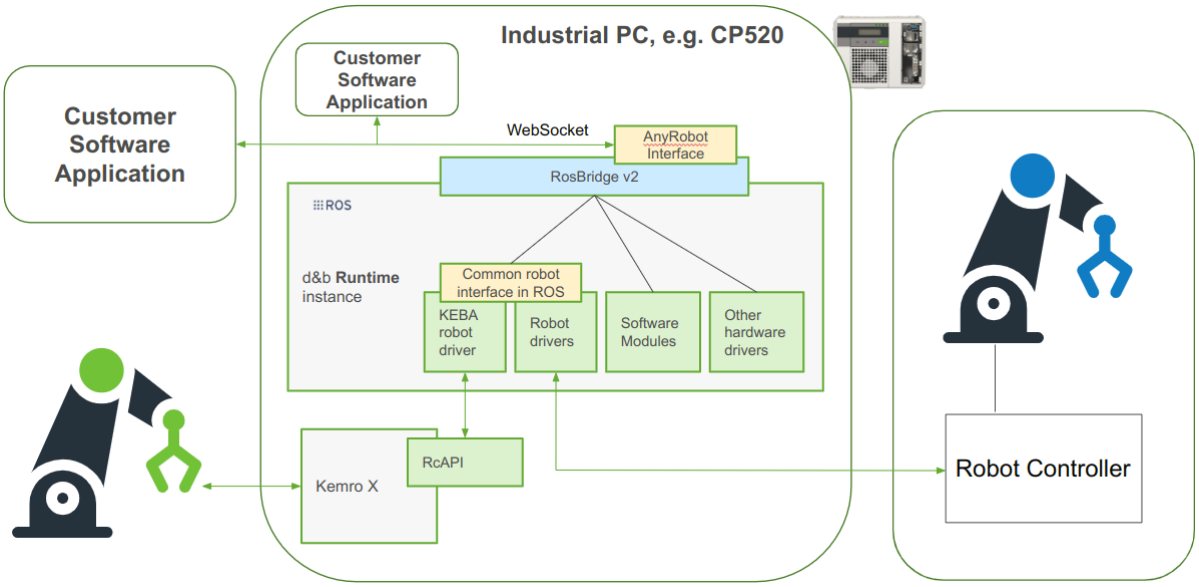


Figure 5.4: Communication architecture of the implemented system, illustrating the interaction between the planning software, the drag&bot runtime environment, and the robot controller (adapted from [24]).

precisely. Any deviation from this order results in a mismatch of the generated hash, causing the runtime system to reject the connection.

Listing 5.1 presents an illustrative Python implementation of the authentication token computation described in Equation 5.1, following the drag&bot AnyRobot Interface specification [24].

```python
import hashlib
import os

secret = os.getenv('RUNTIME_SECRET')

id_string = 'library'
dest = 'interface'
nonce = 'example_nonce'
timestamp = 0
role = 'user'

raw = secret + id_string + dest + nonce + str(timestamp) + role

mac = hashlib.sha512(raw.encode()).hexdigest()

auth_msg = {
    'operation': 'auth',
    'mac': mac,
    'nonce': nonce,
```

```
    'timestamp': timestamp,
    'role': role
}
```

Listing 5.1: Illustrative Python implementation of SHA-512-based authentication token computation following the drag&bot AnyRobot Interface specification.

Figure 5.5 shows the robot control panel used for system interaction and monitoring.

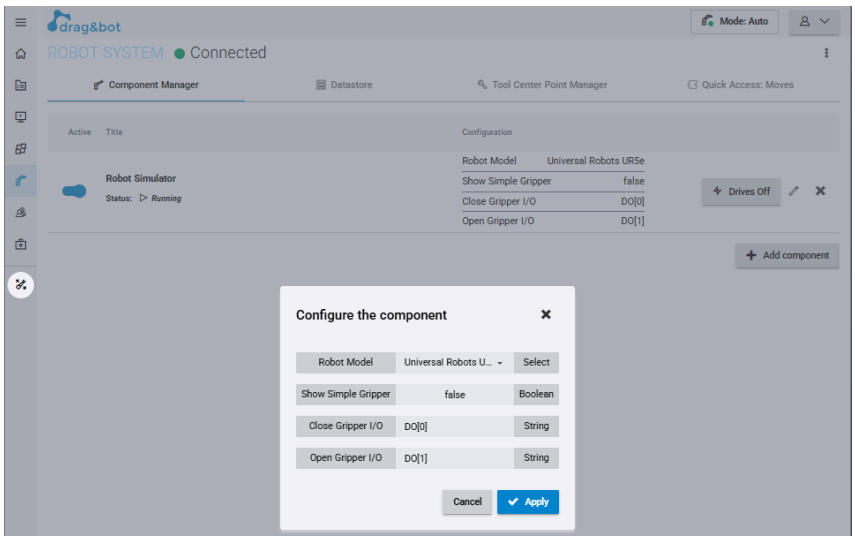


Figure 5.5: Robot control panel interface used for system operation and monitoring.

5.2.5 Configuration Architecture and YAML File Structure

The YAML configuration file serves as the central entry point for all process-related parameters and provides a flexible mechanism for adapting the system to different workpieces without requiring modifications to the underlying source code, thereby fulfilling NFR-02. At the beginning of each session, the file is parsed once, and the resulting configuration data is stored as a dictionary that is shared across all modules in Layers 2 to 4. The structure of the file is organized into six clearly defined top-level sections, each corresponding to a specific functional aspect of the system architecture, which supports clarity and maintainability. YAML is selected over JSON primarily due to its support for inline comments. This allows engineers to document the meaning and physical relevance of individual parameters directly within the configuration file, eliminating the need to consult external documentation. To ensure robustness and security, the configuration is loaded using a safe parsing method, which prevents the execution of unintended code in the event that the file is modified or corrupted. A key design decision concerns the handling of spatial consistency during the regrip process. Rather than defining approach and placement poses independently, all such poses are derived relative to a single reference pose

defined for the orientation table. For example, the approach position is calculated by applying a fixed vertical offset to the placement pose. This design ensures that any change in the position of the orientation table requires only a single modification within the configuration file. In contrast, storing these poses independently could introduce inconsistencies: if the table position is updated but the corresponding approach position is not, the system may execute motions based on outdated spatial data. Such inconsistencies are particularly problematic because they may not be detected during configuration validation, as each parameter remains individually valid despite being collectively incorrect.

Listing 5.2 presents a generalised YAML configuration illustrating the key process, motion, and safety parameters without exposing system-specific or confidential information.

```yaml
# =========================
# Robot configuration (generalised)
# =========================

robot:
  host: <robot_ip_address>
  port: <port_number>
  speed_mps: 0.5
  accel_percent: 20
  blend_percent: 15
  io_group: DO
  io_pin: 1

# =========================
# Process and sheet definition
# =========================

process:
  sheet:
    length_m: 0.4
    width_m: 0.3
    thickness_m: 0.005
    mass_kg: 2.5

  grip:
    cup_count: 2
    min_edge_clear_m: 0.05
    min_spacing_m: 0.2
```

Listing 5.2: Generalised YAML configuration illustrating robot parameters, process definition, and motion and safety constraints.

5.2.6 Operator interface – interactive 3d sheet bending viewer

Operator input for each execution cycle is provided through an interactive three-dimensional sheet bending viewer developed in this work (hereafter referred to as the *3D viewer*), rather than a conventional text-based interface. This design decision is primarily motivated by the spatial nature of bend side selection. By presenting the workpiece directly as a 3D model, the operator can identify relevant surfaces and orientations more intuitively than by relying on abstract side labels. In practice, this reduces the likelihood of incorrect selections while keeping the interaction simple, as the operator performs only visually guided inputs and the system handles the underlying computations.

The 3D viewer is implemented as a web-based application using Plotly Dash and renders the workpiece as an interactive mesh derived from the corresponding STL file. It operates as a local service and is integrated into the drag&bot Operator Cockpit through the Custom URL panel mechanism. A new panel of type *Custom URL* is added to the dashboard via the dashboard configuration interface, where the local viewer address is specified as the target. Once configured, the viewer is rendered as an embedded panel within the cockpit, allowing the operator to interact with the 3D mesh and associated controls without leaving the drag&bot environment. The configuration is stored as part of the dashboard definition and remains available across sessions, ensuring a consistent setup for repeated operation.

This integration ensures that configuration, monitoring, and execution are performed within a single interface, eliminating unnecessary context switching and providing a more coherent workflow from the operator's perspective. The overall system setup used during development and validation, including the robot, press brake, and orientation table, is shown in Figure 5.6.

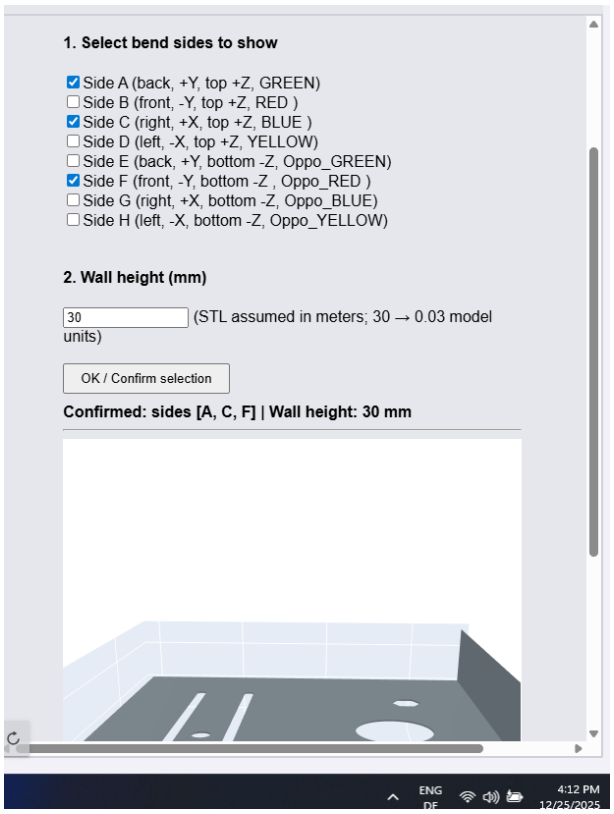


Figure 5.6: Virtual simulation environment used for implementation and validation. The setup includes the robot, press brake, orientation table, and sheet workpiece. Source: own work.

To accommodate different levels of operator expertise, the interface is divided into two modes. In basic mode, the operator selects from a predefined set of bending sides and specifies the required wall height. The system automatically prevents invalid combinations, particularly those that would require additional operations such as regripping, which are intentionally excluded at this level to simplify interaction. In contrast, expert mode exposes additional parameters, including geometric constraints, tool configuration, and pose definitions. However, changes made in this mode are not stored automatically and must be explicitly transferred to the configuration file, ensuring that persistent modifications are introduced in a controlled manner.

In parallel with user input, the system performs a geometric analysis of the imported mesh to identify internal features such as holes. These features are detected based on their orientation and their spatial relationship to the outer boundary of the sheet, and are subsequently classified according to their geometric characteristics. The resulting feature information is used during grip planning to exclude unsuitable regions from consideration. The workpiece model used for this analysis is illustrated in Figure 5.7. Additionally, the workpiece mass is estimated using a standard material density, providing a conservative reference for evaluating whether a given gripping configuration is physically feasible.

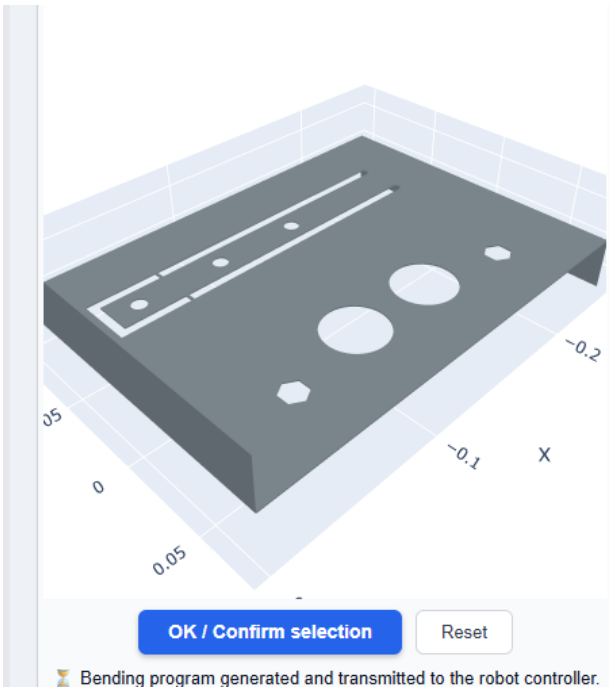
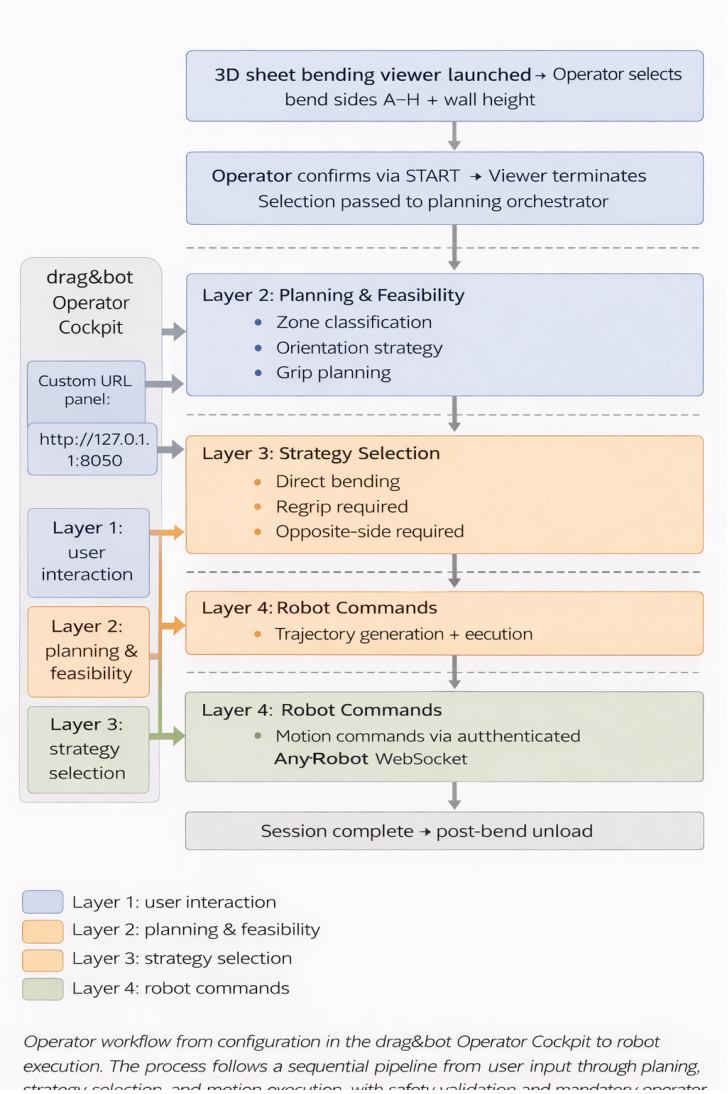


Figure 5.7: Sheet metal workpiece model used for geometry analysis. Internal cut-outs are detected and excluded from the grip candidate search region. Source: own work.

Once the operator confirms the selection, the defined parameters are transmitted to the planning module in a structured format. The viewer then terminates after a short delay. This behaviour is intentional, as it prevents further interaction while the robot is executing a trajectory, thereby avoiding inconsistencies that could arise if configuration changes were introduced during active motion.

The complete sequence of operator interaction and system processing is summarised in Figure 5.8, which illustrates the progression from initial configuration through planning, strategy selection, and final execution.



Operator workflow from configuration in the drag&bot Operator Cockpit to robot execution. The process follows a sequential pipeline from user input through planing,

Figure 5.8: Operator Workflow .Every gate must be passed before the next begins.All operator interactions occu within the drag and bot enviornment. *Source: own work.*

5.3 Development and simulation Environment

With the architectural foundation established in Section 5.1, the implementation requires a defined environment in which all modules can be executed, tested, and validated before involving any physical robot. This section outlines the environment, including the drag and bot virtual simulation platform, the UR5e robot model with its TCP configuration, and the spatial zone classification used to divide the workspace and determine the execution path for each bending session.

5.3.1 Simulation Platform, Robot Model, and TCP Configuration

The drag and bot platform is used as the primary environment for development, integration, and testing within this thesis, as it combines a virtual robot controller

and the AnyRobot WebSocket Interface within a single system, eliminating the need for additional simulation tools or direct access to physical hardware [24]. All Python-based modules are implemented and validated within this environment. The platform operates on the same Ubuntu system as the planning software, enabling communication through the local loopback interface and avoiding timing variations caused by external network conditions. The planning software establishes a single authenticated WebSocket connection to ws://<local-ip>:9091, which is used to handle all command types, including motion execution, IO operations, object manager interactions, and operator HI dialog requests. All session-related parameters are loaded from operator input during initialization and are explicitly passed to each module through function arguments. This approach ensures that dependencies remain transparent at the interface level and prevents modules from accessing outdated values from shared contexts. The same principle is consistently applied across all twenty-seven modules, with the configuration structure described in Section 5.2.5.

Within the simulation environment, the Universal Robots UR5e is selected as the manipulator model, as it corresponds to the robot used in the physical KEBA press brake system and is supported by a validated kinematic representation in the drag and bot platform [24]. The UR5e provides six degrees of freedom, a maximum reach of approximately 850 mm, and a payload capacity of 5 kg, which are sufficient for handling the sheet metal workpieces considered in this implementation [30]. All motion targets generated by the planning software are evaluated against this kinematic model, allowing reachability limitations to be identified during simulation rather than during operation on the physical system.

The Tool Centre Point applied to all motion targets is defined through the drag and bot tool management entry project_3_tcp, in accordance with SR-03 in Section 3.2.4 and based on a system requirement specified by the company. This ensures a consistent reference for tool geometry so that all generated poses correspond to the actual gripper configuration. The TCP is defined using the ZYX Euler convention with the following offsets relative to the robot flange frame: x = 0.000 m, y = 0.034 m, z = 0.194 m, RZ = −90.0°, RY = −75.0°, RX = 0.0°. The Z offset of 194 mm represents the distance from the flange to the suction contact surface, while the Y offset of 34 mm reflects the asymmetric arrangement of the suction cups along the gripper. All coordinates computed by the planning software are defined at this TCP, allowing the drag and bot runtime to derive the corresponding flange pose through inverse transformation without requiring the planning layer to explicitly consider flange geometry. The TCP identifier project_3_tcp is stored in operator input, enabling flexible adaptation of the system. If adjustments are required in the future, the TCP can be updated by the user through parameter changes in the operator interface, without requiring modifications to the source code.

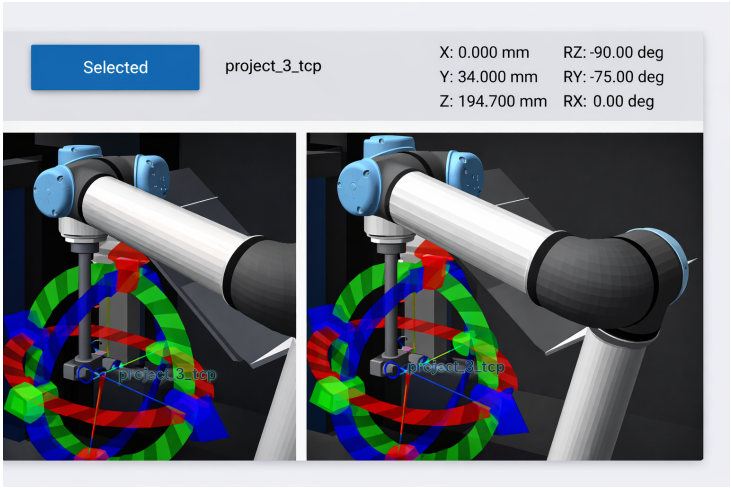


Figure 5.9: TCP of UR5e robot. *Source: own work.*

5.3.2 Virtual Cell Layout and Spatial Zone Classification

The drag and bot simulation cell includes the UR5e robot, the press brake model, the orientation table, and a sheet metal workpiece whose tracked pose (primary tracked sheet object) forms the basis for all geometric calculations in the planning software. In addition, a secondary sheet object is registered to support two-holder operation, allowing the system to track whichever workpiece is active during execution. All fixture positions, workspace limits, and zone thresholds defined in operator input module, which is aligned with the physical cell layout, ensuring that the simulation results correspond directly to the real system configuration.

At the start of each bending session, the spatial zone classifier retrieves the six-degree-of-freedom pose of the sheet from the object tracking service in the robot world coordinate frame. The origin of this frame is located at the robot base, with the X axis defined laterally and the Y axis longitudinally across the cell. Based on this reference, the XY position of the sheet centroid is evaluated using a threshold-based rule to assign the workpiece to one of four spatial zones. This process is performed automatically using live tracking data without operator input. The classification is defined relative to the robot base position, which must be correctly specified in the operator input configuration.

The XY plane is divided into four regions around the robot base. A centroid with negative X and positive Y is assigned to Zone 1 (rear-left), while negative X and negative Y correspond to Zone 2 (front-left). Positive X combined with positive Y defines Zone 3 (rear-right), and positive X with negative Y defines Zone 4 (front-right). The classification thresholds are set to ±10 mm in the X direction and ±50 mm in the Y direction. These asymmetric limits reflect the physical cell geometry and account for increased tracking uncertainty along the longitudinal axis [1]. Zones 3 and 4 lie within the primary reachable workspace of the UR5e and therefore allow direct execution of the pickup and bending sequence from the nom-

inal home configuration. In contrast, Zones 1 and 2 are in a region with limited reachability, requiring an automatic repositioning phase. In this phase, the robot moves to an automatically calculated predefined intermediate configuration within the reachable workspace, after which the standard bending sequence is executed. This repositioning is triggered by the classified zone and is performed without operator intervention. A subsequent tracking update verifies that the sheet lies within the reachable region before execution continues.

If the sheet centroid lies within the dead-band region defined by $|x| < 10$ mm or $|y| < 50$ mm, the classifier returns to an unresolved state. This occurs near the coordinate axes, where small tracking variations may lead to inconsistent zone assignment. To prevent incorrect execution, the system terminates the session and issues blocking notification instructing the operator to reposition the workpiece before restarting.

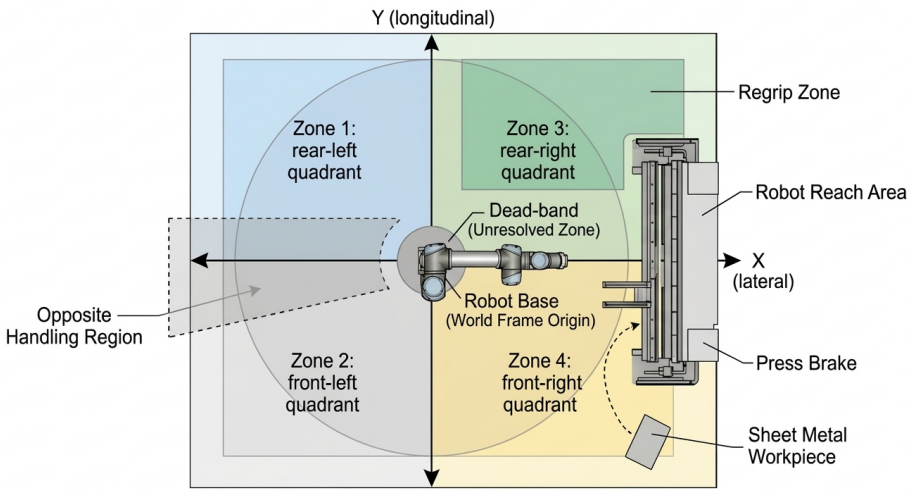


Figure 5.10: Zone classification for robot workspace segmentation. *Source: own work.*

Having defined the simulation environment, robot model, TCP configuration, and zone classification, the following sections describe the functional subsystems in the execution order of a bending session, starting with safety validation, followed by orientation determination, grip planning, and trajectory generation.

Algorithm 1 summarises the zone classification procedure in an abstract form. The classification is based on the sign of the centroid coordinates in the robot world frame, with a threshold-based dead-band region around the origin to prevent unstable assignments due to tracking noise. Implementation-specific details are omitted to comply with confidentiality constraints.

Algorithm 1 Spatial zone classification based on sheet centroid position

Retrieve sheet centroid coordinates (x, y) in the robot world frame
if $|x| < x_{\text{th}}$ **or** $|y| < y_{\text{th}}$ **then**
    Return *unresolved zone*
else if $x < 0$ **and** $y > 0$ **then**
    Assign Zone 1
else if $x < 0$ **and** $y < 0$ **then**
    Assign Zone 2
else if $x > 0$ **and** $y > 0$ **then**
    Assign Zone 3
else
    Assign Zone 4
end if

5.4 Pre-Execution Safety Validation

Safety validation is carried out before any motion is generated to ensure that unsafe conditions and infeasible configurations are identified prior to robot execution, in accordance with NFR-03, NFR-04, and NFR-05. The system incorporates three complementary mechanisms to achieve this: a fixture poses deviation check, a blocking operator confirmation dialog, and a runtime command error watcher. In addition, all trajectory waypoints are generated automatically and verified using robot kinematics to confirm reachability and feasibility before execution, preventing invalid motion commands from being issued.

5.4.1 Fixture Pose Deviation Validation

The fixture pose deviation check retrieves the current poses of six critical fixtures from the drag&bot object manager and compares them with reference poses obtained during the initial calibration of the cell. The monitored elements include the primary sheet object at the press brake, the finished-part position on the regrip table base and support, and the placement and attachment references at the flip station. These fixtures are treated as fixed references within the planning software, since all grip positions, trajectory waypoints, and placement operations are computed under the assumption that their positions remain unchanged. Any deviation from these reference poses can therefore result in motion targets that are geometrically valid but physically incorrect.

The deviation check is applied uniformly across all spatial zones. Each fixture is evaluated using both positional and orientation criteria. The positional deviation is computed as the Euclidean distance

$$\Delta_{pos} = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2} \tag{5.2}$$

and the orientation deviation is defined as

$$\Delta_{ang} = \max\left(|\Delta RZ|, |\Delta RY|, |\Delta RX|\right). \tag{5.3}$$

A fixture is considered out of tolerance when Δ_{pos} exceeds 5 mm or Δ_{ang} exceeds 0.10 rad (5.7°). The positional threshold is selected to remain below the minimum clearance used in grip planning, ensuring that fixture misalignment does not produce misleadingly valid grip configurations. The angular threshold corresponds to the minimum deviation that would affect the accuracy of the vacuum cup contact on the sheet surface. The primary sheet object is evaluated using the same deviation criteria as all other fixtures. Since the planning software relies on the current sheet pose for all geometric computations, any displacement from the expected reference may lead to incorrect motion generation. Therefore, the sheet is included in the deviation evaluation in the same manner as other fixtures, and any detected deviation is treated as a potential configuration error. If one or more fixtures exceed the defined thresholds, the system generates a structured warning that lists all affected fixtures using their generic labels, together with the measured positional and angular deviations. This warning is presented through a blocking HI dialog titled *Fixture position warning*, which allows the operator to either continue with explicit caution or cancel the operation for further inspection. The operator's decision is recorded, as the responsibility for accepting out-of-tolerance conditions remains with the human operator. If a fixture is not detected in the object manager response, it is treated as missing and is automatically flagged regardless of zone classification.

5.4.2 Operator Authorization and Runtime Error Monitoring

The operator confirmation dialog is positioned as the first step in the planning orchestrator startup sequence, preceding all subsequent processes including fixture validation, visualization, and motion command transmission. The dialog enforces a mandatory decision: either initiate the automated workflow or cancel execution. This behavior is structurally enforced, as the orchestrator does not proceed until a response is received from the HI service. Consequently, execution cannot begin without explicit operator authorisation, ensuring compliance with safety requirements. If the operator selects cancel, the orchestrator terminates without issuing any command to the robot runtime.

In parallel with this confirmation step and all subsequent execution phases, a motion error watcher operates as a background subprocess. This component subscribes to the `/command_result` ROS topic through an independent WebSocket connection, enabling continuous monitoring of command execution outcomes. A separate connection is required to maintain an uninterrupted receive loop without blocking the orchestrator's command flow. When a non-zero result code is detected, indicating a motion execution failure, the watcher immediately triggers a modal HI

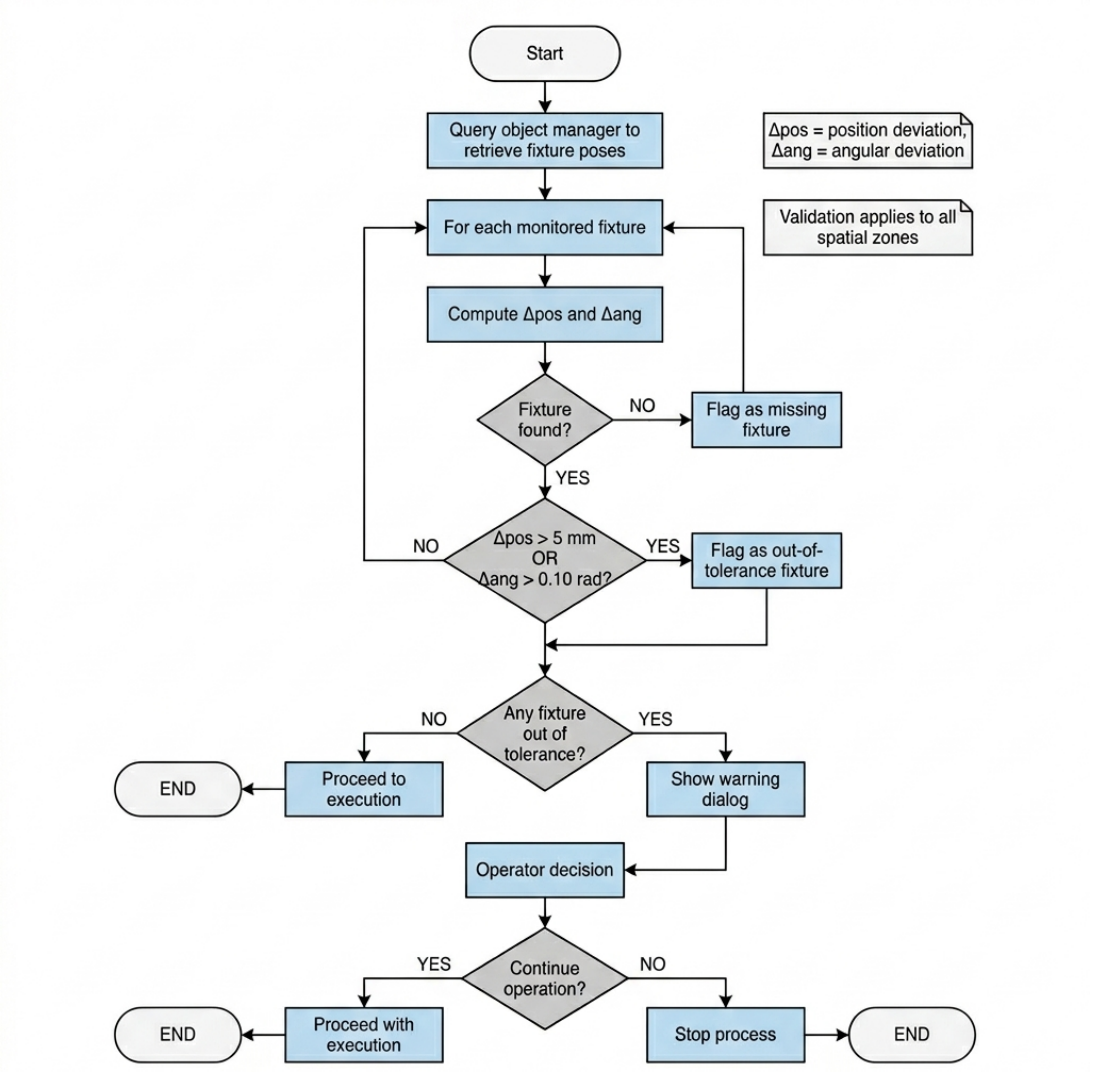


Figure 5.11: Fixture pose deviation validation workflow. Each monitored fixture is evaluated by comparing positional and angular deviations against defined thresholds. Out-of-tolerance or missing fixtures trigger a blocking warning dialog requiring operator confirmation. Validation is applied uniformly across all spatial zones. Source: own work.

notification to inform the operator. This ensures that execution errors are identified and communicated in real time.

The error watcher does not directly interrupt or terminate the orchestrator. Instead, it provides immediate feedback through the HI interface, requiring operator acknowledgement before further interaction is permitted. This design ensures that execution anomalies do not pass unnoticed, while maintaining operator authority over subsequent actions.

Together with the fixture pose deviation check, these mechanisms establish a layered safety structure. The confirmation dialog enforces operator authorization prior to execution, while the error watcher ensures that failures during execution are promptly detected and communicated. Both mechanisms utilize the drag&bot

HI service, providing a consistent interface for all safety-related interactions. With these safeguards in place, the subsequent section addresses the workpiece orientation strategy for the bending process.

5.5 Workpiece Orientation Strategy

Following pre-execution safety validation, the first geometric step in each bending cycle is the determination of a valid pickup orientation for the sheet. This is necessary because the sheet may be placed in arbitrary orientations within the workspace, while only a subset of these orientations is kinematically feasible for the target bending operation. The orientation strategy evaluates the tracked sheet pose, applies a structured selection process, and produces a validated pickup pose for subsequent grip planning.

5.5.1 Zone-Based RZ Admissibility

The admissible range of the sheet rotation about the Z-axis is defined by a zone-dependent window $[RZ_{\text{min}}, RZ_{\text{max}}]$. This constraint reflects the limited rotational capability of the robot end-effector within different regions of the workspace. Since the reachable orientation range varies with the spatial position of the sheet, zone classification is performed prior to orientation evaluation. The corresponding admissibility window is retrieved from the configuration, ensuring that only feasible orientations are considered. This approach avoids selecting poses that appear valid geometrically but are unreachable during execution.

5.5.2 Pickup Orientation Validation

Before applying the orientation selection strategy, the tracked pose is validated to ensure that the gripper approaches the sheet from above. This is evaluated using the product $\cos(RX) \cdot \cos(RY)$. If the value is below -0.10, the tool Z-axis is oriented downward and no modification is required. Otherwise, a corrective transformation is applied:

$$RZ' = \text{wrap}(RZ + 180^\circ), \quad RY' = -RY, \quad RX' = \text{wrap}(RX + 180^\circ) \tag{5.4}$$

This transformation ensures that the gripper is correctly aligned with the sheet surface without altering the pickup position.

5.5.3 Three-Level Orientation Selection Strategy

To ensure that a valid pickup orientation can be determined for any initial placement, a three-level selection strategy is employed. The strategy is evaluated sequentially

in a strict priority order, and execution terminates as soon as a valid orientation satisfying the admissibility constraints is identified. This approach guarantees robustness against arbitrary initial sheet orientations while maintaining preference for geometrically consistent and kinematically favorable configurations.

The workpiece is defined with four reference faces established during system commissioning, each associated with a fixed angular offset about the sheet's local vertical axis. These faces provide a structured reference for interpreting the tracked orientation. The offsets correspond to $0°$, $180°$, $+90°$, and $-90°$ relative to the reference face. By compensating for these offsets, the system can normalise any detected orientation into a common reference frame, enabling consistent evaluation across different initial conditions.

Level 1: Reference-Face Equivalent Orientation At the highest priority level, the tracked orientation is converted into its reference-face equivalent by removing the face-specific angular offset. This is achieved by applying an inverse rotation about the sheet's local Z-axis:

$$R_{\text{new}} = R_{\text{raw}} \cdot R_z(-d) \tag{5.5}$$

where d represents the angular offset associated with the detected face. The resulting rotation matrix is then decomposed into ZYX Euler angles using:

$$R = R_z(\alpha) \cdot R_y(\beta) \cdot R_x(\gamma) \tag{5.6}$$

$$RZ = \text{atan2}(R_{10}, R_{00}), \quad RY = \text{arcsin}(-R_{20}), \quad RX = \text{atan2}(R_{21}, R_{22}) \tag{5.7}$$

The computed RZ value is then evaluated against the zone-specific admissibility window $[RZ_{\text{min}}, RZ_{\text{max}}]$. If the orientation lies within this range, it is selected as the pickup pose. This level is preferred because it aligns all configurations to a common reference frame, resulting in more predictable motion generation and reduced trajectory complexity.

Level 2: Detected-Pose Evaluation If the reference-face equivalent orientation falls outside the admissible range, the strategy proceeds to evaluate the raw tracked orientation directly. This level accounts for cases where the sheet is already positioned within a valid orientation relative to the robot workspace, and the reference-face conversion would unnecessarily shift it outside the admissible window.

The raw RZ value is therefore checked against the same admissibility limits. If it satisfies the constraint, the raw pose is accepted as the pickup orientation. This level

preserves valid initial configurations without enforcing additional transformations.

Level 3: Inverted-Pose Fallback If both the reference-face equivalent and the raw orientation fail to satisfy the admissibility condition, a fallback orientation is generated by applying a 180° inversion:

$$RZ' = \mathsf{wrap}(RZ \pm 180^\circ), \quad RY' = -RY, \quad RX' = -RX \tag{5.8}$$

This transformation effectively rotates the sheet about its vertical axis while reversing its orientation about the horizontal axes. The inverted pose expands the set of reachable configurations and ensures that a pickup orientation can always be produced, even when the initial pose lies outside the calibrated admissible region.

However, since this orientation may not strictly satisfy the predefined admissibility constraints, it is marked with an *admissibility unconfirmed* flag. This flag is propagated to downstream modules, including the grip planner and trajectory generator, allowing them to apply additional safety margins or conservative motion constraints when executing the operation.

Algorithm 2 High-level orientation selection strategy

```
Retrieve tracked sheet pose and active zone
Load zone-specific admissibility window {R_Z^min, R_Z^max}
Validate pickup orientation and apply downward-tool correction if required
Compute reference-face equivalent orientation
if reference-face equivalent satisfies admissibility window then
    Select reference-face orientation
    Set admissibility status to confirmed
else if detected orientation satisfies admissibility window then
    Select detected orientation
    Set admissibility status to confirmed
else
    Compute inverted orientation
    Select inverted orientation
    Set admissibility status to unconfirmed
end if
Forward selected orientation and admissibility status to downstream planning modules
```

Algorithm 2 summarises the implemented orientation strategy at an abstract level. It reflects the execution order of the internal software pipeline without exposing implementation-specific source code, which is omitted due to confidentiality constraints.

Strategy Outcome The three-level strategy guarantees that a pickup orientation is always determined while prioritising configurations that are both kinemati-

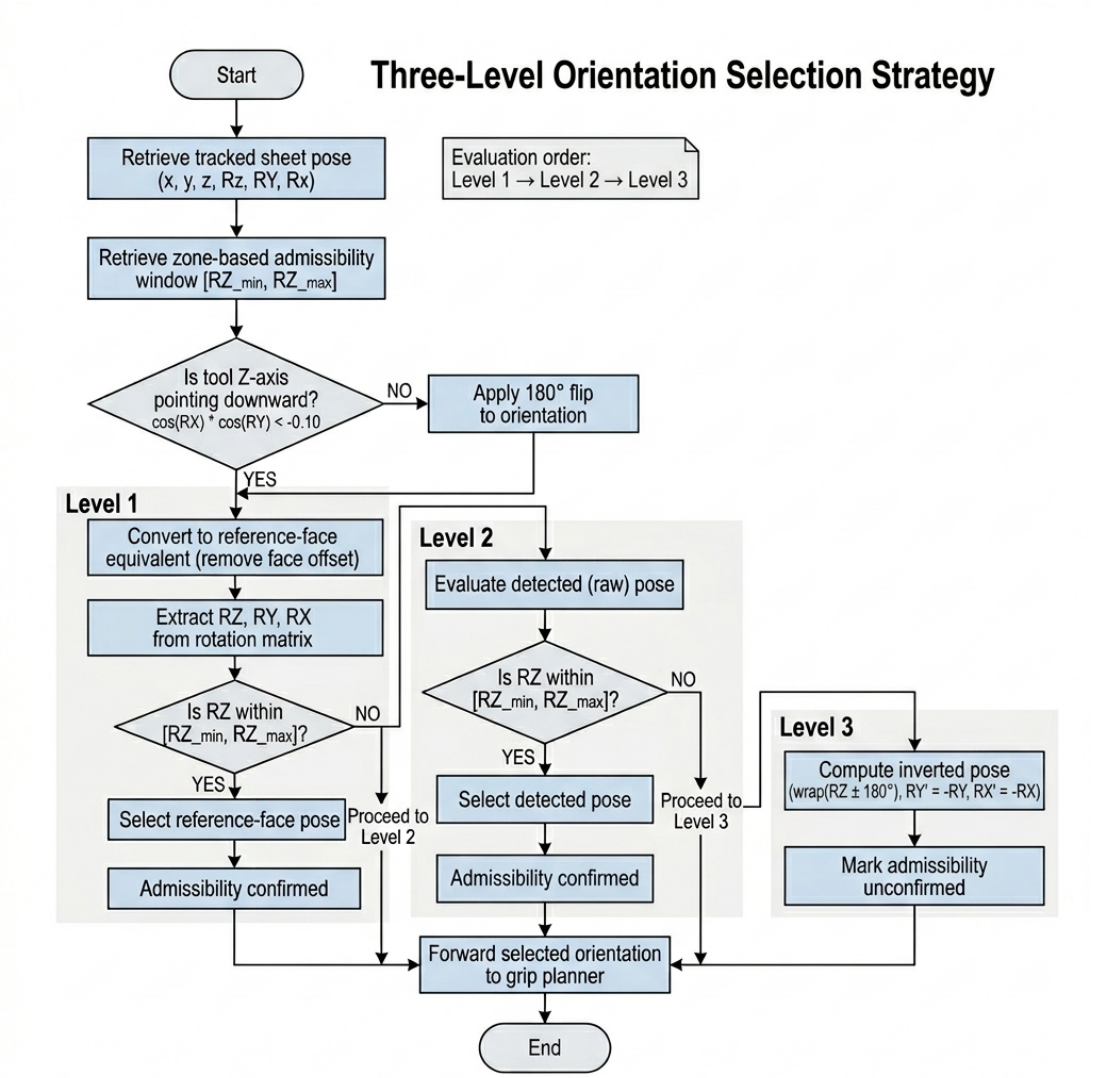


Figure 5.12: Three-level system architecture illustrating the hierarchical structure of the implementation. *Source: own work.*

cally feasible and geometrically consistent. By enforcing a strict evaluation order and incorporating a controlled fallback mechanism, the approach balances robustness and operational safety. The final selected orientation, together with its admissibility status, is forwarded to subsequent planning stages for execution.

5.6 Workpiece Handling and Gripper Control

Once a valid pickup orientation has been determined, the system must establish a reliable physical interaction between the gripper and the sheet while maintaining consistency between the physical robot state and the virtual simulation. Any mismatch between the vacuum state of the gripper and the attachment state in the simulation would lead to incorrect pose propagation and invalid trajectory computation in subsequent stages. This consistency is ensured through two coordinated mechanisms: digital IO control of the vacuum gripper and kinematic attachment

within the simulation environment

5.6.1 Vacuum Gripper Digital IO Sequence

The vacuum gripper is actuated using two digital output signals on the UR5e controller, mapped to DO[0] and DO[1] in the drag&bot IO management system. DO[0] activates the vacuum by closing the suction valve, while DO[1] releases the vacuum by opening the exhaust valve. These signals are strictly mutually exclusive; at no point are both outputs activated simultaneously, as this would result in conflicting pressure states and potential hardware damage.

Control commands are issued via the `/robot/set_digital_ios` service of the Any-Robot Interface, which accepts structured IO commands specifying the target pin and its Boolean state. To ensure reliable gripping, a dwell period of 250 ms is applied after activating the vacuum. This delay allows sufficient time for the suction cups to establish a stable seal with the sheet surface[31] . Initiating further operations before this stabilization would result in a virtual attachment being applied to a physically unstable grasp. A secondary delay of 150 ms is introduced after each IO operation to ensure that the state transition propagates through the ROS communication layer before subsequent commands are executed. This prevents race conditions between command execution and system state updates.

5.6.2 Kinematic Object Attachment and Detachment

In the simulation environment, the physical coupling between the gripper and the sheet is represented through kinematic attachment. When the gripper successfully grasps the sheet, the corresponding object is attached to the robot's tool frame, ensuring that all subsequent robot motions are reflected in the sheet's pose. Conversely, when the vacuum is released, the object is detached and remains fixed in its last known position.

Attachment is performed using the `/dnb_object_manager/attach` service without enforcing a position override. This avoids introducing discontinuities between the tracked object pose and the simulated state, as the tracker may not exactly match the instantaneous tool frame position. Position override is reserved exclusively for controlled initialization procedures where explicit alignment is required.

Detachment is executed through the `/dnb_object_manager/detach` service using the same reference frame. To maintain consistency, attachment and detachment operations are always executed in a fixed sequence together with the corresponding IO commands:

- Grasp sequence: vacuum ON $\rightarrow$ dwell $\rightarrow$ attach
- Release sequence: vacuum OFF $\rightarrow$ dwell $\rightarrow$ detach

This sequence is encapsulated within centralized helper routines that are invoked by all modules performing grasp or release actions. Centralizing this logic ensures that the physical and virtual states remain synchronised at all times and prevents invalid states where the vacuum is active without a corresponding attachment, or vice versa.

5.7 Grip Position Planning and Feasibility Validation

With a valid pickup orientation and gripper control mechanism established, the system must determine feasible suction cup placement on the sheet surface. Arbitrary placement is not acceptable, as it may violate geometric constraints or fail to provide sufficient holding force during transport. The grip planner therefore evaluates both geometric and physical constraints to ensure a stable and executable grip configuration.

5.7.1 Sheet Geometry and Hole Detection

The sheet is modelled as a rectangular surface with uniform mass distribution, defined by its length, width, thickness, and mass from the configuration. When the mass is not explicitly provided, it is estimated from the sheet volume and the configured material properties.

In addition to the outer boundary, circular hole regions are identified from the STL mesh using the geometry analysis stage described previously. Each hole is represented by its center position and radius in the sheet coordinate frame. These hole regions are incorporated into grip planning so that candidate cup positions overlapping any hole are excluded, since such placements would prevent the formation of a reliable vacuum seal and would therefore compromise workpiece retention during transport.

5.7.2 Vacuum Holding Force Validation

For each candidate grip configuration, the planner evaluates whether the available vacuum force is sufficient to retain the sheet under gravitational and dynamic loading conditions. The contact area of a single suction cup is defined as

$$A_{\text{cup}} = \pi \left(\frac{d}{2}\right)^2 \tag{5.9}$$

The available vacuum holding force across n cups is then given by

$$F_{\text{vacuum}} = n \cdot \Delta P \cdot A_{\text{cup}} \cdot \eta \tag{5.10}$$

The required holding force is computed as

$$F_{\text{required}} = SF \cdot m \cdot \sqrt{a_{xy}^2 + (g + a_z)^2} \tag{5.11}$$

A configuration is considered feasible only when $F_{\text{vacuum}} \geq F_{\text{required}}$. This validation ensures that only physically stable grip configurations are passed to the subsequent candidate evaluation stage.

5.7.3 Grip Candidate Search and Selection

Candidate suction cup positions are generated using a multi-resolution grid-based search over the sheet surface. The search begins with a coarse discretization and progressively refines candidate positions in feasible regions to improve placement accuracy. In addition to grid-based sampling, supplementary candidate positions are generated near hole boundaries to maximize usable gripping area while maintaining clearance constraints.

Each candidate position is evaluated against geometric constraints. The cup center must maintain a minimum distance from sheet edges, avoid overlapping with hole regions including the configured clearance buffer, and satisfy spacing constraints in the case of multi-cup configurations. In the two-cup configuration, candidate positions are evaluated as pairs, and the spacing between cup centers is constrained to ensure an adequate grip span across the workpiece.

All candidates are first filtered using geometric constraints and vacuum holding feasibility before being evaluated by a scoring function. The scoring considers edge clearance, hole clearance, and center bias, while in the two-cup configuration the span between cups is additionally maximized to improve stability during transport. The implemented search procedure is summarized in Algorithm 3.

Algorithm 3 Grip position planning procedure

```
Load sheet geometry and gripping parameters
Validate vacuum holding feasibility
Generate candidate positions on the sheet surface
Filter candidates using geometric constraints
if two-cup configuration then
    Evaluate candidate pairs
end if
Score feasible candidates
Select best configuration
Compute A1, B1, R1 waypoints
Forward result to trajectory generation
```

The highest-scoring candidate configuration is selected as the final grip position. Based on this position, three key waypoints are computed: the approach pose (A1), the attachment pose (B1), and the retreat pose (R1). These waypoints are

derived from the selected grip position together with the sheet surface height and tool geometry offsets, ensuring a consistent and collision-free pickup sequence.

Figure 5.13 illustrates the overall grip planning pipeline, including force validation, candidate generation, geometric filtering, pair evaluation for the two-cup case, scoring, and waypoint derivation.

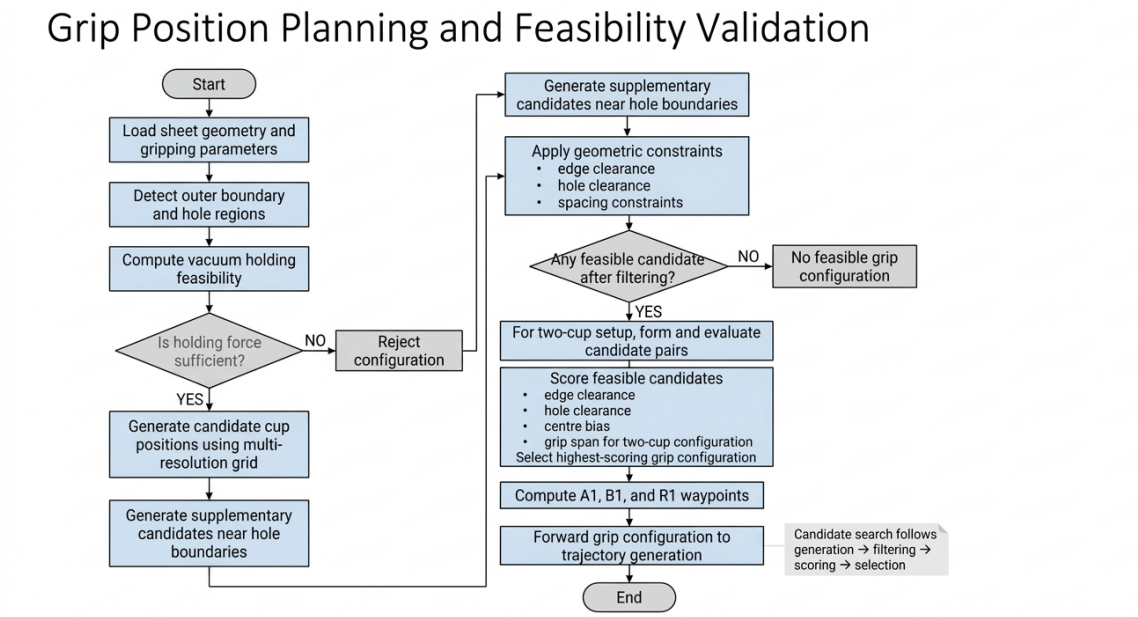


Figure 5.13: Grip position planning and feasibility validation workflow. The process begins with the loading of sheet geometry and gripping parameters, followed by vacuum holding feasibility validation. Candidate cup positions are then generated on the sheet surface, filtered using geometric constraints, and evaluated according to the configured scoring criteria. In the two-cup configuration, feasible candidates are additionally assessed as pairs before the highest-scoring grip configuration is selected. The final grip position is used to derive the approach, attachment, and retreat waypoints forwarded to trajectory generation. Source: own work.

5.8 Regrip Planning and Execution

Even when a valid pickup orientation and a feasible grip position are established, the current gripper configuration may not allow the robot to reach the target bending pose. This occurs when the required flange pose lies outside the robot workspace, or when kinematic constraints such as joint limits or singularity proximity prevent a feasible motion solution. To address this, the system implements an automatic regrip planning strategy that determines the required reorientation dynamically. This strategy directly addresses the limitations identified in the State of the Art by enabling automatic determination of regrip requirements within an industrially applicable framework.

5.8.1 Regrip Decision Strategy

In contrast to conventional approaches that rely on predefined regrip steps, the proposed system evaluates a discrete set of candidate workpiece orientations and selects a configuration that satisfies kinematic feasibility. This enables the system to determine both the required orientation and the number of regrip steps automatically.

The candidate set consists of discrete orientation increments that correspond to physically achievable rotations on the orientation table. For each candidate orientation, the system performs the following sequence:

- Update the workpiece pose according to the candidate orientation
- Determine a valid pickup orientation using the orientation strategy described in Section5.5
- Compute feasible grip positions using the grip planner described in Section 5.7
- Evaluate kinematic feasibility using inverse kinematics and trajectory validation

A candidate orientation is accepted when all feasibility checks are satisfied. The evaluation follows a heuristic selection strategy, where the first feasible orientation is selected to ensure computational efficiency suitable for real-time execution. The number of required regrip steps is implicitly determined based on the difference between the current orientation and the selected feasible orientation. This approach avoids unnecessary reorientation operations while ensuring that the final configuration is executable within the robot workspace.

Algorithm 4 Regrip Orientation Selection

```
Define candidate orientation set
for each candidate orientation do
    Compute updated workpiece pose
    Evaluate pickup orientation
    Compute grip configuration
    Check kinematic feasibility
    if feasible then
        Select orientation
        break
    end if
end for
Determine required number of regrip steps
```

Algorithm 4 summarizes the automatic decision process. The evaluation is performed using the existing planning pipeline, ensuring consistency with orientation determination, grip planning, and trajectory generation.

5.8.2 Regrip Execution Sequence

Once a feasible orientation is selected, the system executes a structured regrip sequence to physically realize the required reorientation. The execution is based on a six-step cycle designed to ensure safe and consistent handling of the workpiece:

1. Transport the workpiece from the bending station approach position to the orientation table deposit position while maintaining the current grip.
2. Release the workpiece by deactivating the vacuum and performing kinematic detachment.
3. Move the robot to a retract standoff position to ensure collision-free clearance.
4. Re-approach the workpiece at the updated orientation and perform the grasp sequence.
5. Repeat the regrip cycle if multiple orientation adjustments are required.
6. Perform the final grasp and return the workpiece to the bending station.

Each motion segment is executed using smooth trajectory profiles to minimize dynamic loads on the vacuum gripper and ensure stable handling. Intermediate standoff positions are used to guarantee collision-free transitions between grasping and releasing operations.

5.8.3 System Integration and Configuration

The regrip execution sequence operates within the existing motion framework and uses configuration-defined target positions, including the deposit position, re-grasp position, retract standoff, and bending station approach position. These parameters define the physical layout of the cell but do not determine the number of regrip steps.

The decision to trigger a regrip is handled by the planning orchestrator, which evaluates the feasibility of the current configuration based on orientation admissibility and kinematic constraints. The separation between decision-making and execution ensures modularity and allows the regrip planning strategy to be extended without modifying the motion execution layer.

Compared to configuration-driven approaches, the proposed method improves adaptability by determining the minimum required number of regrip steps dynamically. At the same time, it maintains computational efficiency by evaluating a structured set of candidate orientations rather than performing full global optimization. This makes the approach suitable for industrial deployment, where predictable and robust behavior is required.

5.9 Opposite-face Bending Workflow

After completing all primary-side bending operations, the system evaluates whether additional bends are required on the opposite face of the sheet. These opposite-side bends cannot be executed directly from the initial pickup configuration, as the required face is not accessible to the press brake tool. Therefore, the sheet must be reoriented through a controlled flip operation before continuing the bending process.

5.9.1 Pre-Flip Verification and Transition Correction

Before initiating the flip sequence, the system verifies that the current sheet pose matches the expected starting configuration. This verification is performed using a wrap-safe six-degree-of-freedom pose comparison. A position tolerance of 2 mm and an angular tolerance of 1° define the acceptable deviation.

This check is necessary because the sheet may have undergone regrip operations during the primary bending sequence. If the current pose deviates from the expected reference, subsequent motions would be computed from an incorrect starting configuration.

If the verification fails, a transition correction step is executed. The system first restores the reference orientation using the orientation strategy described in Section 5.5. It then performs a controlled pickup and repositioning sequence to re-establish a consistent and verified coupling between the robot and the sheet.

If the verification is successful, the correction step is skipped, and the system proceeds directly to the flip sequence.

5.9.2 Sheet Flip Sequence and Opposite-face bending Execution

The flip operation is performed using a dedicated flip station within the robotic cell. The sheet is transferred through two predefined reference positions to achieve physically feasible reorientation.

First, the robot transports the sheet to the flip station placement position while maintaining the current grasp. The sheet is then released in a controlled manner. The robot subsequently moves to an intermediate position and re-approaches the sheet from the opposite side, establishing a new grasp that results in the flipped orientation.

This two-stage transfer is used instead of a direct in-gripper rotation. A direct rotation would require a large continuous motion that is not kinematically feasible within the robot workspace and may violate collision constraints.

After the flip operation, the sheet is oriented such that the previously inaccessible face is now aligned with the press brake tool. The system then applies the same validation pipeline used for primary-side bending. This includes zone classification, orientation determination, grip position planning, and trajectory generation.

Using the same validation hierarchy ensures consistent behaviour across both primary and opposite-side operations. It also avoids introducing separate handling logic for different sheet faces.

Figure- 5.14 illustrates the complete opposite-side bending workflow, including pre-flip verification, optional transition correction, flip execution, and subsequent bending operations.

After completing all opposite-side bends, any remaining staged operations are executed using the existing multi-stage handling framework. The system then proceeds to the final unloading sequence.

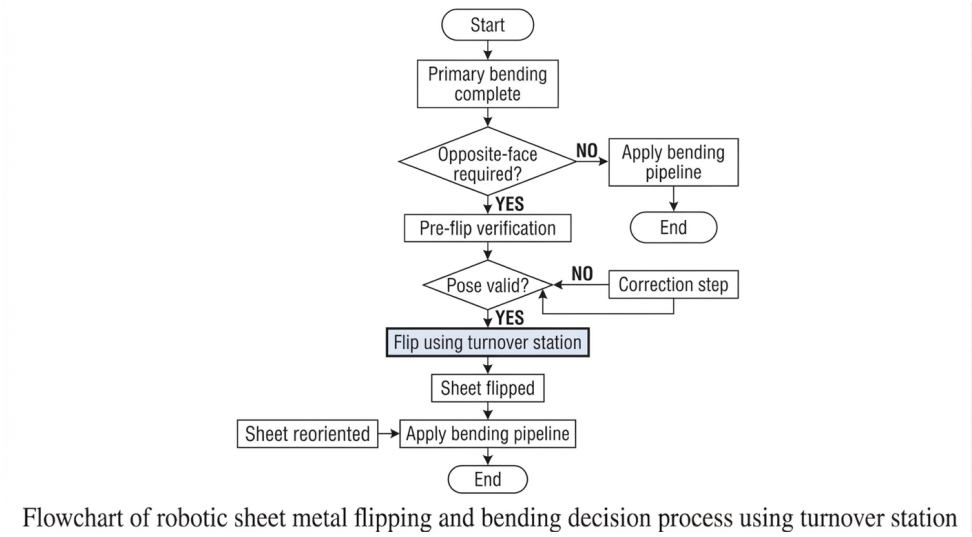


Figure 5.14: Opposite face flip operation for regrip orientation adjustment

5.10 Trajectory Generation

Trajectory generation transforms a validated target pose into executable robot motion. The input to this stage is an ordered sequence of tool centre point (TCP) poses obtained from upstream planning modules such as grip planning, regrip execution, and opposite-face handling. Each pose defines the desired position and orientation of the end-effector in the robot world frame. The trajectory generator connects these poses through a smooth and feasible motion that can be executed by the robot controller.

Rather than executing a direct transition between poses, the system constructs a continuous trajectory represented by intermediate waypoints. These waypoints define the path followed by the end-effector and ensure smooth motion from the start to the target configuration.

5.10.1 Three-Phase LIN Trajectory Structure

Each trajectory is constructed using a three-phase structure: lift, lateral travel, and descent. In the lift phase, the TCP is raised vertically to a predefined safe height to ensure clearance from the workpiece and surrounding environment. In the lateral travel phase, the robot moves towards the target position at this safe height. During this phase, the orientation is constrained to limit the RY angle, reducing proximity to wrist singularities. In the descent phase, the TCP moves from the safe height to the final target pose, applying the required orientation for task execution.

To separate orientation changes from contact motion, an intermediate rotation pose is introduced above the target. This ensures that large orientation adjustments are completed before the final descent, improving kinematic feasibility near the target.

5.10.2 Waypoint Generation and Interpolation

The trajectory between successive poses is generated through interpolation in Cartesian space. For a start pose $\mathbf{T}_s$ and a target pose $\mathbf{T}_t$, the translational component is interpolated using

$$\mathbf{p}(\lambda) = (1 - \lambda)\mathbf{p}_s + \lambda\mathbf{p}_t \tag{5.12}$$

where $\lambda \in [0, 1]$ is the interpolation parameter. The orientation is interpolated using spherical linear interpolation (SLERP) in quaternion space, ensuring smooth rotational motion along the trajectory [32].

To achieve smooth velocity transitions, the interpolation parameter follows a minimum-jerk profile

$$s(u) = 10u^3 - 15u^4 + 6u^5 \tag{5.13}$$

which produces zero velocity and acceleration at the start and end of each motion segment [33].

By sampling this progression at discrete intervals, a sequence of intermediate TCP waypoints is generated that approximates the continuous trajectory [8].

```
def min_jerk(u):
    return 10*u**3 - 15*u**4 + 6*u**5

waypoints = []
for k in range(N):
    u = k / (N - 1)
    s = min_jerk(u)
    p = (1 - s) * p_start + s * p_target
    r = slerp(r_start, r_target, s)
```

```
10      waypoints.append((p, r))
```

Listing 5.3: Illustrative waypoint generation using minimum-jerk interpolation

The number of waypoints depends on the path length and a predefined step size, ensuring a balance between trajectory smoothness and computational efficiency.

```
1 distance = euclidean_dist(p_start, p_target)
2 N = max(2, math.ceil(distance / delta_s_max))
3 samples = [k / (N - 1) for k in range(N)]
```

Listing 5.4: Illustrative determination of waypoint count based on path length

5.10.3 Kinematic Mapping and Joint Execution

Each interpolated TCP waypoint is converted into a corresponding robot configuration using inverse kinematics. This mapping determines the set of joint angles required to achieve the desired end-effector pose. For each waypoint, a feasible joint solution is selected based on continuity with the previous configuration, ensuring smooth transitions and avoiding abrupt joint changes.

As the robot executes the sequence of configurations, all six joints evolve continuously. This produces a smooth end-effector motion that follows the interpolated Cartesian path from the initial pose to the final target pose.

5.10.4 Workspace Constraints and Singularity Handling

To ensure feasibility, each waypoint is evaluated within the robot's workspace. The TCP pose is transformed into the corresponding flange pose, and the position is checked against the cylindrical workspace limits defined by maximum reach and vertical bounds. If a waypoint exceeds these limits, it is projected onto the nearest valid boundary before further processing.

Singularity avoidance is addressed during the lateral travel phase by limiting the allowable orientation range. This prevents the robot from entering configurations that would result in unstable joint motion or excessive joint velocities. The full target orientation is applied only near the final approach, where the robot is in a more stable configuration.

5.10.5 Motion Execution and Path Reduction

The interpolation process produces a dense set of waypoints. To ensure compatibility with the robot controller, the trajectory is reduced to a manageable number of points by retaining only those that exceed predefined positional or rotational thresholds. This preserves the shape of the trajectory while reducing communication overhead.

The resulting waypoint sequence is transmitted to the robot controller as a series of linear motion commands. The controller executes these commands using motion blending, allowing smooth transitions between consecutive waypoints without stopping at each intermediate position. As a result, the robot follows a continuous trajectory that approximates the planned TCP path.

The overall trajectory generation process forms a deterministic pipeline: validated TCP poses are interpolated into intermediate waypoints, mapped into joint configurations through inverse kinematics, adjusted for workspace constraints, and executed by the robot as a continuous motion from the start to the target pose.

5.11 System Integration, Execution Pipeline, and Conclusion

The developed system operates as an integrated pipeline that transforms operator-defined bending inputs into fully executed robot motion. This pipeline coordinates multiple modules, including task planning, trajectory generation, and execution control, ensuring consistent and deterministic system behaviour.

The execution begins with the operator defining the bending configuration, including the selection of bend sides and associated parameters. These inputs are processed by the planning module, which determines the sequence of operations required to complete the bending task. This stage establishes the order of primary-side bends, identifies necessary regrip operations, and determines whether opposite-face handling is required.

Following planning, the system enters the primary bending phase. Each bend is executed sequentially using validated motion strategies. For every operation, a feasible grip position is determined, orientation constraints are applied, and a corresponding trajectory is generated. Regrip sequences are performed when required to maintain stable and feasible grasp configurations throughout the process.

If bending operations on the opposite face are requested, the system transitions to the flip workflow. The sheet is transferred through the flip station, reoriented, and reattached so that the previously inaccessible face becomes available. The same planning and execution principles used in the primary phase are then applied, ensuring consistent handling across both sides of the workpiece.

All robot motion within these stages is executed through the trajectory generation pipeline described in Section 5.10. Each planned TCP pose is converted into intermediate waypoints, mapped to joint configurations through inverse kinematics, and executed as continuous motion using linear interpolation and blending. This ensures smooth and stable end-effector movement throughout the bending process.

After completion of all bending operations, the system performs the post-process unload as part of the execution pipeline. The robot attaches to the finished workpiece, transports it to the designated buffer station, and releases it in a controlled manner. The robot then moves to a predefined safe start pose, restoring the system

to a known configuration and ensuring safe operator access to the workspace.

The overall system therefore follows a structured execution pipeline:

- Operator input and bend configuration
- Planning of bending sequence and regrip strategy
- Execution of primary-side bending operations
- Regrip handling between operations
- Opposite-face handling via flip workflow (if required)
- Trajectory generation and motion execution
- Post-process unload and system reset

5.11.1 Conclusion

The presented system demonstrates a fully integrated approach to automated sheet metal bending using robotic manipulation. By combining structured task planning, constraint-aware trajectory generation, and controlled execution, the system achieves consistent and reliable handling of complex bending sequences.

The integration of regrip strategies and opposite-face handling enables the system to process multi-stage bending operations without manual intervention. The use of interpolation-based trajectory generation ensures smooth and stable motion, while workspace and orientation constraints maintain kinematic feasibility throughout execution.

Overall, the system provides a robust and deterministic framework for automated bending operations, supporting flexible task configurations while maintaining safety, repeatability, and execution stability. This integration forms the foundation for further evaluation through system reliability and failure analysis in the following section.

5.12 System Reliability and Failure Mode Analysis

To evaluate the robustness of the developed robotic bending system, a Failure Mode and Effects Analysis (FMEA) is conducted. This analysis identifies potential failure modes within the system, evaluates their impact on operation, and considers mitigation strategies implemented in the system design.

FMEA is a structured approach used to assess system reliability by analysing three key factors: severity (S), occurrence (O), and detectability (D). The combined effect of these factors is quantified using the Risk Priority Number (RPN), defined as

$$\text{RPN} = S \times O \times D \tag{5.14}$$

where each parameter is rated on a qualitative scale, typically from 1 (low) to 10 (high). Higher RPN values indicate higher priority risks that require mitigation.

5.12.1 Identified Failure Modes

The analysis focuses on key operational stages of the system, including gripping, trajectory execution, regrip handling, flip operations, and final placement. The identified failure modes are summarised in Table 5.2.

Table 5.2: Failure Mode and Effects Analysis (FMEA) of the robotic bending system

Failure Mode	Effect	S	O	D	RPN	Cause	Mitigation
Sheet unreachable during pickup	Robot stops mid-trajectory; session aborted	7	3	3	63	Invalid spatial placement or orientation outside admissible zone	Spatial zone classification and orientation constraints applied before motion; system halts with operator notification
Incorrect grip position (cup over hole)	Vacuum failure; sheet may fall during transport	8	2	2	32	Incomplete feature detection in hole map	Clearance offsets applied around detected holes; final validation ensures no suction cup overlaps with holes
Vacuum holding force insufficient	Sheet slips or drops during motion	8	2	1	16	Insufficient gripping force due to load or acceleration	Force feasibility check applied for each candidate grip; safety factor enforced and invalid grips rejected
Motion command rejected by controller	Robot stops; process interruption	6	3	2	36	Invalid or inconsistent motion command format	Command validation and structured message formatting ensure consistent communication with controller
Trajectory exits workspace boundary	Motion infeasible; execution halted	6	4	3	72	Target pose near or beyond reachable workspace limits	Workspace clamping applied to all trajectory points; limits enforced on radial and vertical reach
Inverse kinematics failure due to excessive attachments	Robot cannot compute valid configuration	5	2	3	30	Multiple simultaneous object attachments causing solver overload	Strict attach-detach discipline enforced; each attachment preceded by release of previous objects
Self-collision in simulation	Invalid motion path generated without warning	4	3	5	60	Lack of self-collision detection in simulation environment	Three-phase trajectory structure reduces collision risk; limitation acknowledged for physical deployment
Configuration error (invalid or missing parameters)	System fails at startup; no execution	3	3	1	9	Missing or incorrectly defined configuration parameters	Configuration validation performed at startup; errors detected before motion execution

5.12.2 Mitigation Strategies

Several design measures are incorporated to reduce the likelihood and impact of these failures. Grip stability is ensured through controlled attachment sequences and orientation constraints. Trajectory feasibility is maintained through workspace validation and structured motion planning. Singularity risks are reduced by limiting orientation changes during lateral motion and separating rotational adjustments from the final descent phase.

The flip workflow introduces controlled attachment and detachment stages to ensure correct reorientation of the sheet. Additionally, the use of live object tracking for pose acquisition improves positioning accuracy during both bending and unloading operations. The deterministic execution pipeline further reduces uncertainty by enforcing a fixed sequence of validated operations.

5.12.3 Discussion

The FMEA results indicate that the most critical risks are associated with pose accuracy and trajectory feasibility, as reflected by higher RPN values. These risks are mitigated through a combination of constraint-based planning, controlled motion generation, and staged execution.

Overall, the system demonstrates a balanced design in which high-severity failure modes are either low in occurrence or effectively mitigated through detection and control mechanisms. This supports reliable operation of the robotic bending process while maintaining safety and repeatability.

The presented analysis highlights that the integration of planning, trajectory generation, and execution control not only enables automated bending but also contributes significantly to system robustness. This confirms the effectiveness of the proposed system architecture in handling complex multi-stage manipulation tasks.

Bibliography

[1] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*, 2nd ed. London: Springer, 2016, ISBN: 978-1-4471-1500-0.

[2] S. Aomura and K. Koguchi, "Development of a robotic bending system for sheet metal," *Journal of Materials Processing Technology*, vol. 130–131, pp. 362–367, 2002. DOI: 10.1016/S0924-0136(02)00744-1

[3] Z. Pan, J. Polden, N. Larkin, S. van Duin, and J. Norrish, "Recent progress on programming methods for industrial robots," *Robotics and Computer-Integrated Manufacturing*, vol. 20, no. 2, pp. 137–150, 2012. DOI: 10.1016/j.rcim.2003.07.004

[4] J. Krüger, T. K. Lien, and A. Verl, "Cooperation of human and machines in assembly lines," *CIRP Annals*, vol. 58, no. 2, pp. 628–646, 2009. DOI: 10.1016/j.cirp.2009.09.009

[5] C. Schou, J. S. Damgaard, S. Bøgh, and O. Madsen, "Human–robot interface for instructing industrial robots using skill primitives," *Robotics and Computer-Integrated Manufacturing*, vol. 29, no. 6, pp. 403–412, 2013. DOI: 10.1016/j.rcim.2013.06.002

[6] A. Björkelund, J. Malec, K. Nilsson, and P. Nugues, "Knowledge-based industrial robot task planning and execution," *Robotics and Computer-Integrated Manufacturing*, vol. 28, no. 5, pp. 558–568, 2012. DOI: 10.1016/j.rcim.2012.01.003

[7] S. K. Gupta, D. A. Bourne, K. Kim, and S. Krishnan, "Automated process planning for sheet metal bending operations," *Journal of Manufacturing Systems*, vol. 17, no. 5, pp. 338–360, 1998. DOI: 10.1016/S0278-6125(98)80035-2

[8] L. Biagiotti and C. Melchiorri, *Trajectory Planning for Automatic Machines and Robots*. Springer, 2008.

[9] W. Wan and K. Harada, "Regrasp planning using 10,000s of grasps," *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2017.

[10] J. Bohg, A. Morales, T. Asfour, and D. Kragic, "Data-driven grasp synthesis—a survey," *IEEE Transactions on Robotics*, 2014.

[11] X. Liao and G. G. Wang, "Evolutionary path planning for robot-assisted sheet metal bending," *Journal of Manufacturing Systems*, vol. 22, no. 2, pp. 112–127, 2003. DOI: 10.1016/S0278-6125(03)80006-4

[12] J. R. Duflou, D. Van Oudheusden, J.-P. Kruth, and D. Cattrysse, "Modeling of vacuum grippers for the design of energy-efficient vacuum-based handling processes," *CIRP Annals*, vol. 55, no. 1, pp. 17–20, 2006. DOI: 10.1016/S0007-8506(07)60375-0

[13] *Robots and robotic devices — vocabulary*, International Organization for Standardization, 2012.

[14] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2005, ISBN: 978-0-13-123629-5.

[15] T. Lozano-Pérez, "Robot programming," *Proceedings of the IEEE*, vol. 71, no. 7, pp. 821–841, 1983. DOI: 10.1109/PROC.1983.12678

[16] J. Krüger, T. Krüger, et al., "Skill-based programming of industrial robots," *CIRP Annals*, vol. 68, no. 2, pp. 661–684, 2019. DOI: 10.1016/j.cirp.2019.05.002

[17] G. Pahl, W. Beitz, J. Feldhusen, and K. H. Grote, *Engineering Design: A Systematic Approach*, 3rd ed. London: Springer, 2007. DOI: 10.1007/978-1-84628-319-2

[18] Acieta, *Robotic press brake, welding & palletizing cell for cotterman*, 2018. [Online]. Available: https://www.acieta.com/automation-application/robotically-integrated-bending-solutions-ribs/%20(Accessed:%2027%20March%202026)

[19] R. Bogue, "Growth in industrial robot applications," *Industrial Robot: An International Journal*, vol. 45, no. 1, pp. 1–6, 2018. DOI: 10.1108/IR-10-2017-0199

[20] RoboDK, *Robodk: Simulation and offline programming software for industrial robots*, https://robodk.com, Accessed: 27 March 2026, 2024.

[21] G. Chryssolouris, *Manufacturing Systems: Theory and Practice*. Springer, 2013.

[22] F. Tao, Q. Qi, et al., "Digital twin driven smart manufacturing," *Journal of Manufacturing Systems*, vol. 48, pp. 157–169, 2019. DOI: 10.1016/j.jmsy.2018.05.003

[23] V. Villani, F. Pini, F. Leali, and C. Secchi, "Survey on human–robot collaboration in industrial settings," *Mechatronics*, vol. 55, pp. 248–266, 2018. DOI: 10.1016/j.mechatronics.2018.02.009

[24] drag&bot GmbH. "Anyrobot interface documentation." Ulm, Germany, Accessed: Mar. 2, 2026. [Online]. Available: https://docs.dragandbot.com/anyrobot-interface

[25] K. Thramboulidis, "Model-driven engineering for industrial automation systems: A survey," *IEEE Transactions on Industrial Informatics*, vol. 9, no. 4, pp. 1893–1903, 2013. DOI: 10.1109/TII.2013.2251233

[26] Wikipedia contributors, *Euler angles — wikipedia, the free encyclopedia*, [Online; accessed 9-March-2026], 2026. [Online]. Available: https://en.wikipedia.org/wiki/Euler_angles

[27] M. Bär, G. Reinhart, and J. Schilp, "Virtual commissioning of robotic systems: A review and outlook," *Procedia CIRP*, vol. 72, pp. 129–134, 2018.

[28] M. Quigley et al., "ROS: an Open-Source Robot Operating System," in *Proceedings of the ICRA Workshop on Open Source Software*, Kobe, Japan, 2009.

[29] Python Software Foundation, *Python 3 language reference*, Accessed: 2026-03-02, 2024. [Online]. Available: https://docs.python.org/3/

[30] Universal Robots A/S, *UR5e Technical Specifications*, https://www.universal-robots.com/products/ur5-robot/technical-specifications/, Accessed: Mar. 2026, Odense, Denmark, 2026.

[31] J. Mahler, F. Pokorny, B. Hou, M. Roderick, M. Laskey, and K. Goldberg, "Dex-net 3.0: Computing robust vacuum suction grasp targets in point clouds," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017.

[32] G. Wang, F. Xu, K. Zhou, and Z. Pang, "S-velocity profile of industrial robot based on nurbs curve and slerp interpolation," *Processes*, vol. 10, no. 11, p. 2195, 2022. DOI: 10.3390/pr10112195

[33] A.-N. Sharkawy, "Minimum jerk trajectory generation for straight and curved movements: Mathematical analysis," *Advances in Robotics: Reviews*, 2021.